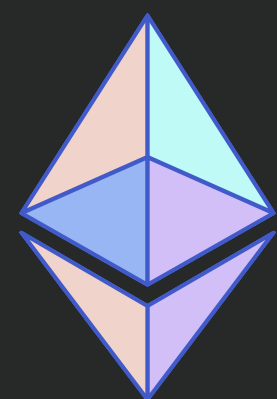


So You Wanna **Post-Quantum** Ethereum transaction signature

Antonio Sanso (asanso )

Researcher - Ethereum Foundation

August 28, 2026



So You Wanna **Post-Quantum** Ethereum transaction signature

Antonio Sanso (asanso )

Researcher - Ethereum Foundation

August 28, 2026

Blockchains components

Execution layer

Transaction signatures,
account security, scripts,
smart contracts

Consensus layer

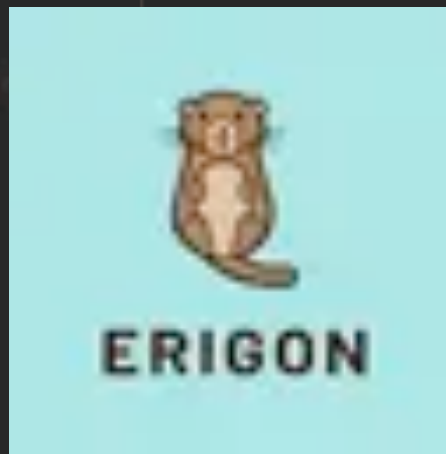
Block production, validator
or miner authentication
finality mechanisms

Data Availability Sampling layer (DAS)

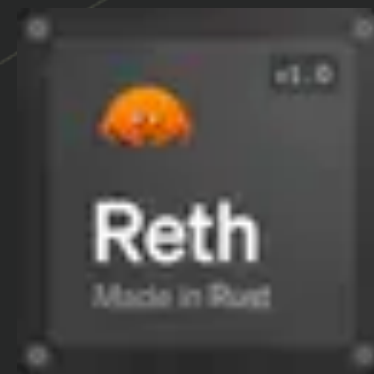
Commitments to
published data, Merkle
trees

Ethereum components

Execution



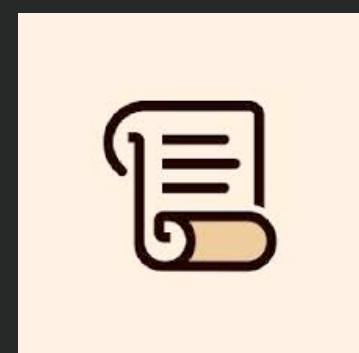
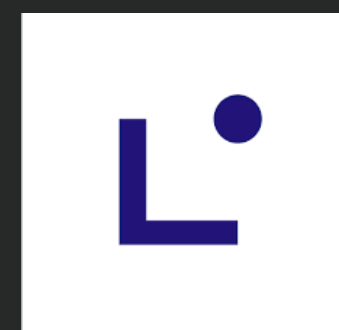
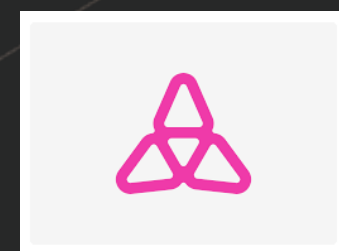
NETHERMIND



Consensus



Data Availability Sampling (DAS)



Blockchains components



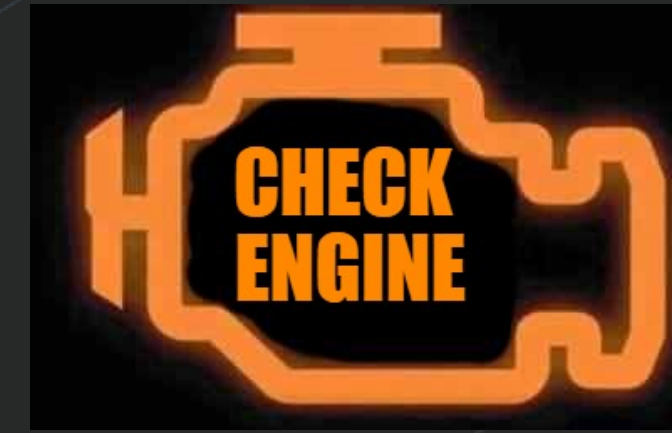
Blockchains components



N.B.: This is not specific to blockchains. Modern digital infrastructure relies on **public key cryptography**, often elliptic curves



Ethereum components



Execution

ECDSA signature
(secp256k1)

Consensus

BLS signature
(BLS12-381)

Data Availability Sampling (DAS)

KZG commitment
(BLS12-381)

Public key Cryptography

Shor's
algorithm - 1994



128-bit Symmetric Keys

**Grover's
algorithm - 1996**



<https://words.filippo.io/128-bits/>

Ethereum goes Post Quantum



Today marks an inflection in the Ethereum Foundation's long-term quantum strategy.

We've formed a new Post Quantum (PQ) team, led by the brilliant Thomas Coratger (@tcoratger). Joining him is Emile, one of the world-class talents behind leanVM. leanVM is the cryptographic cornerstone of our entire post-quantum strategy.

After years of quiet R&D, EF management has officially declared PQ security a top strategic priority. Our journey began in 2019, with the "Eth3.0 Quantum Security" presentation at StarkWare Sessions. Since 2024, PQ has been central to the @leanEthereum vision. The pace of PQ engineering breakthroughs since then has been nothing short of phenomenal.

It's now 2026, timelines are accelerating. Time to go full PQ:

→ **PQ ACD**: Antonio Sanso (@asanso) kicks off a bi-weekly All Core Devs PQ transactions breakout call next month. These sessions focus on user-facing security, covering dedicated precompiles, account abstraction, and longer-term transaction signature aggregation with leanVM.

Jan 2026 [https://
pq.ethereum.org/](https://pq.ethereum.org/)

PQ Ethereum components

Execution

PQTS

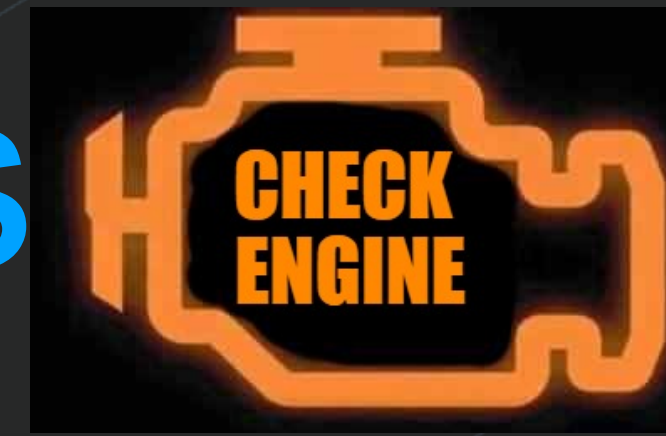
Consensus

Lean Ethereum

Data Availability Sampling (DAS)

LeanDA

Blockchains components

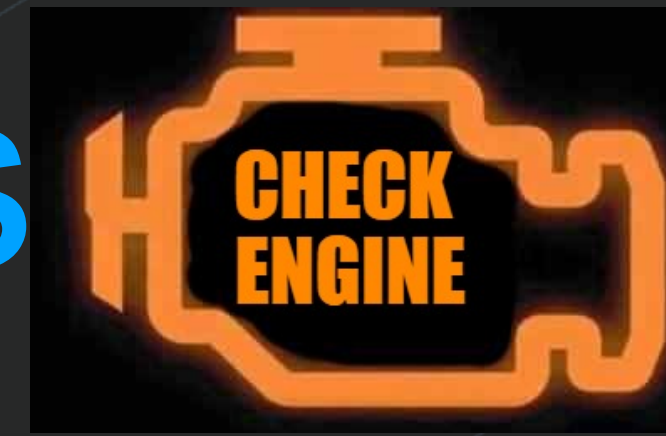


Execution

PQTS



Blockchains components



Execution

PQTS



Breakout room biweekly
call



Mathematic of Post Quantum Cryptography

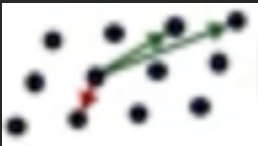
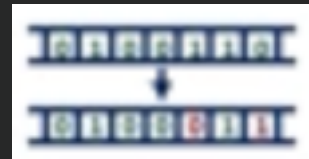
★ Mathematic of Post Quantum Cryptography

- Hash-based cryptography 

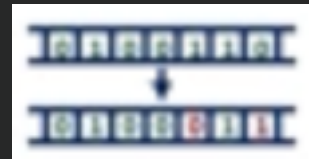
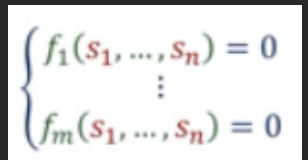
★ Mathematic of Post Quantum Cryptography

- Hash-based cryptography 
- Lattice-based cryptography 
finding short vectors in lattices

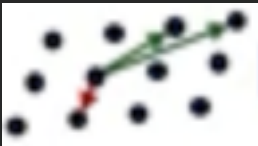
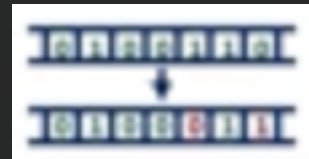
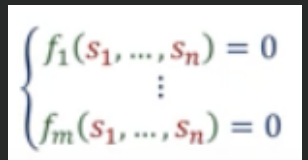
★ Mathematic of Post Quantum Cryptography

- Hash-based cryptography 
- Lattice-based cryptography 
finding short vectors in lattices
- Code-based cryptography 
decoding for random linear codes

★ Mathematic of Post Quantum Cryptography

- Hash-based cryptography 
- Lattice-based cryptography 
finding short vectors in lattices
- Code-based cryptography 
decoding for random linear codes
- Multivariate cryptography 
solving non-linear systems of equations

★ Mathematic of Post Quantum Cryptography

- Hash-based cryptography 
- Lattice-based cryptography 
finding short vectors in lattices
- Code-based cryptography 
decoding for random linear codes
- Multivariate cryptography 
solving non-linear systems of equations
- Supersingular elliptic curve isogeny
cryptography 

Finding isogenies between elliptic curves

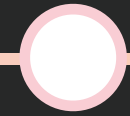
NIST PQ Cryptography Standardization Process

.....it takes time

NIST PQ Cryptography Standardization Process

.....it takes time

2016



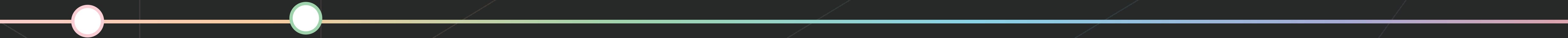
Announce
ment and
Call for
Proposals

NIST PQ Cryptography Standardization Process

.....it takes time

2016

2017

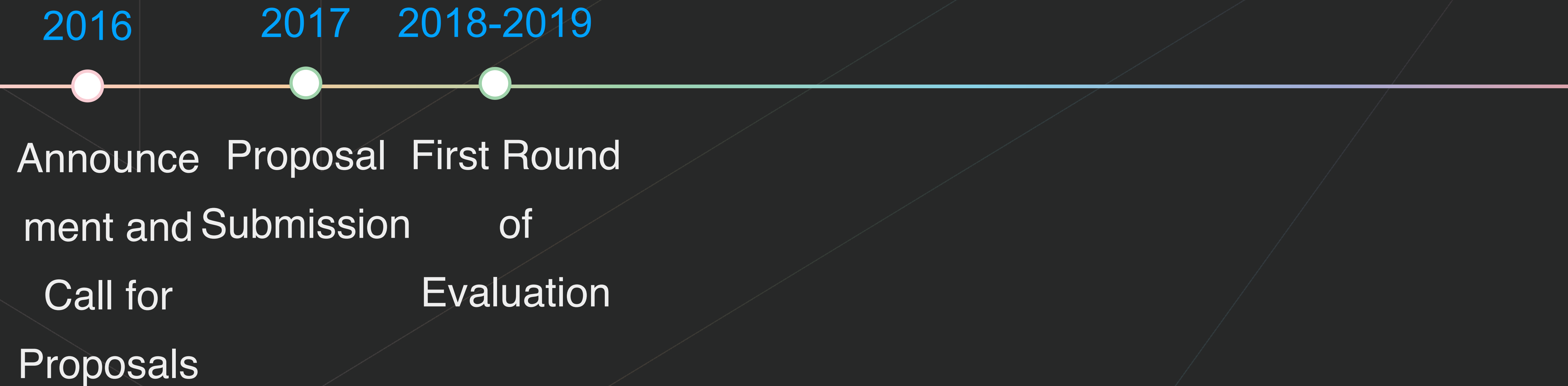


Announce
ment and Submission

Call for
Proposals

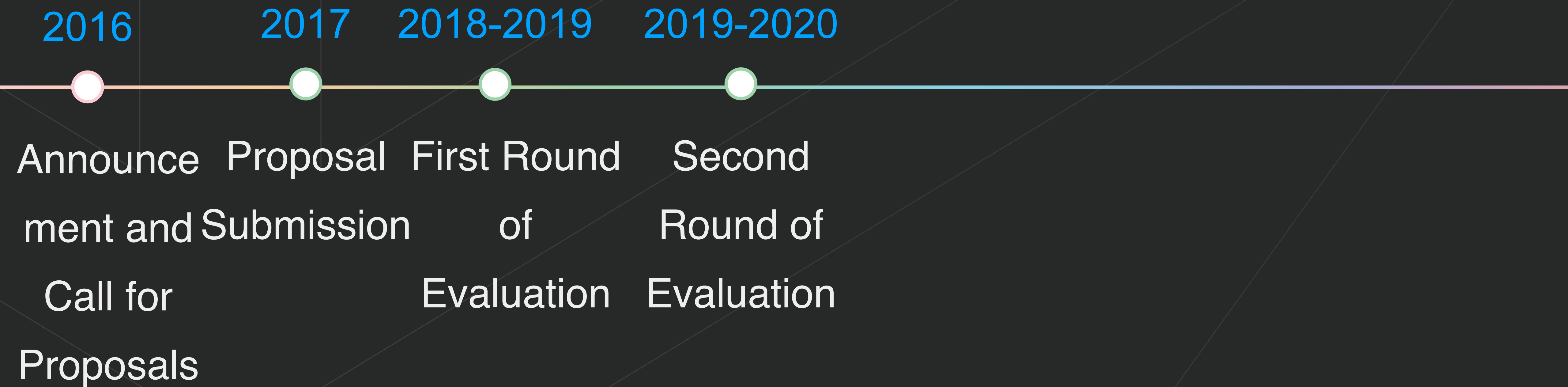
NIST PQ Cryptography Standardization Process

.....it takes time



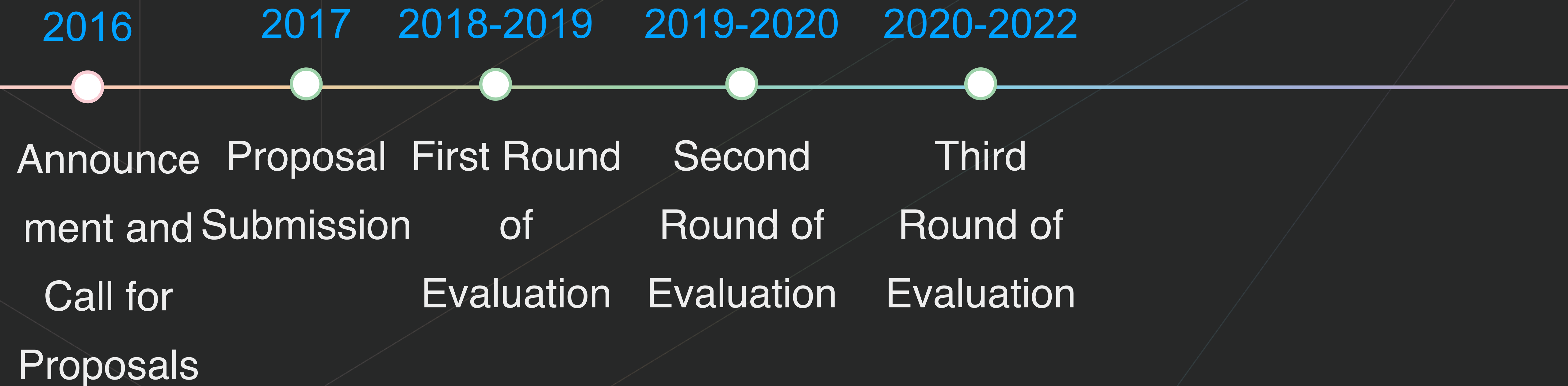
NIST PQ Cryptography Standardization Process

.....it takes time



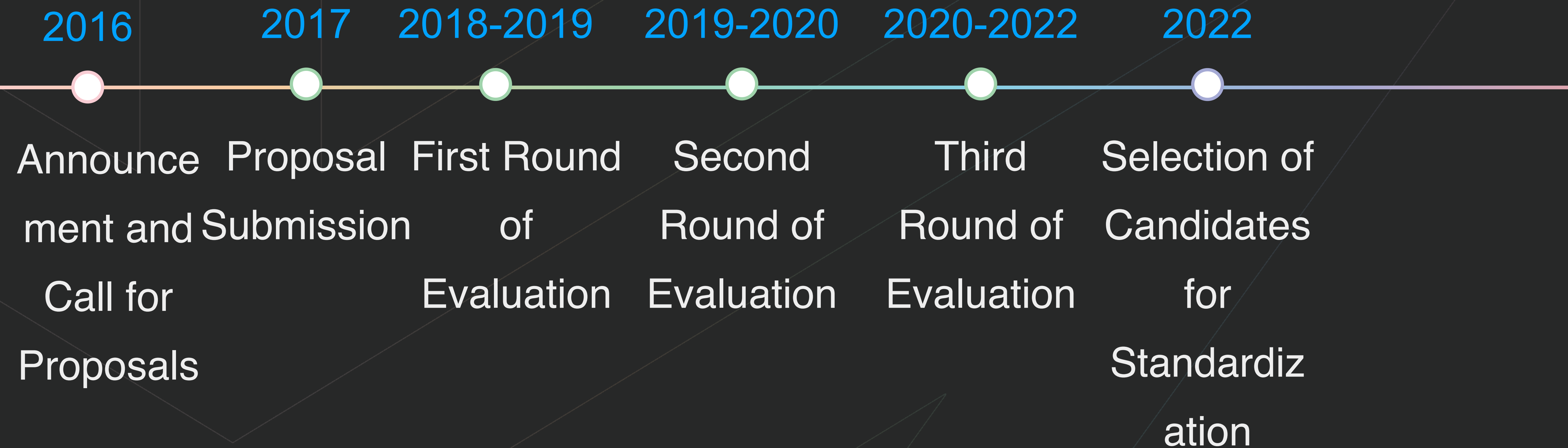
NIST PQ Cryptography Standardization Process

.....it takes time



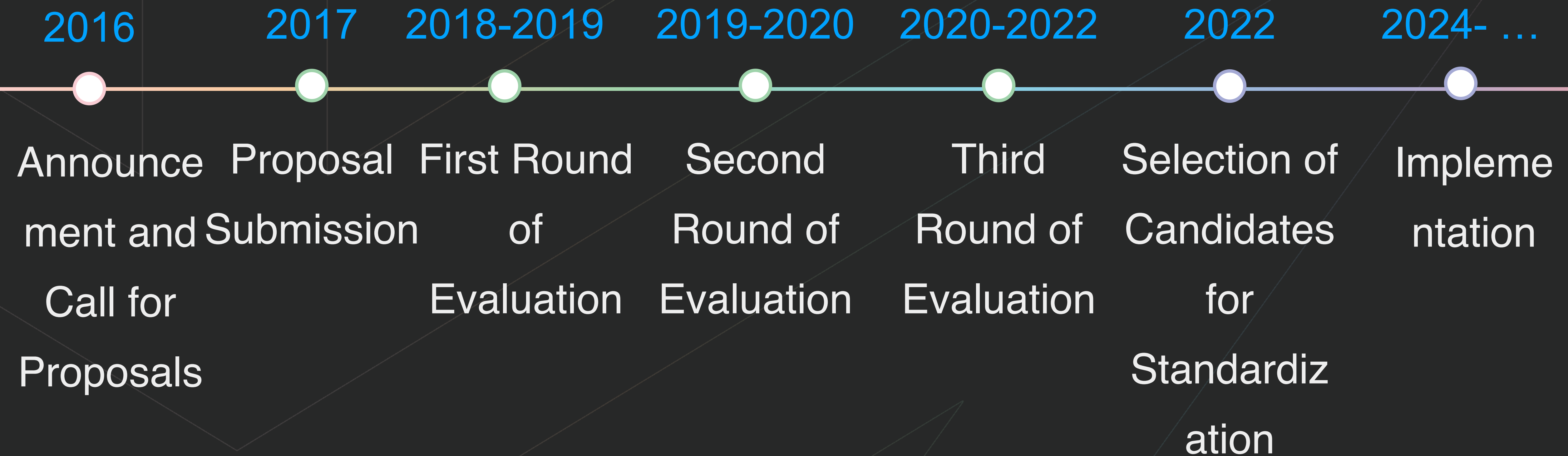
NIST PQ Cryptography Standardization Process

.....it takes time

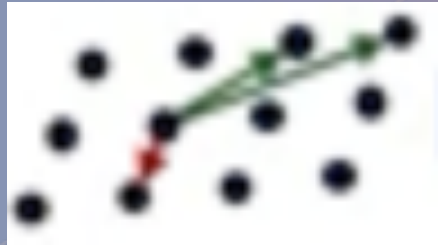
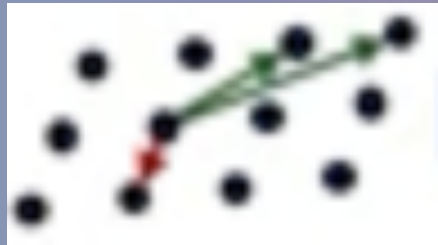


NIST PQ Cryptography Standardization Process

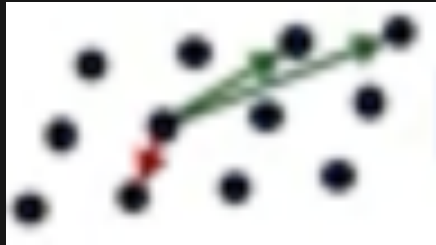
.....it takes time



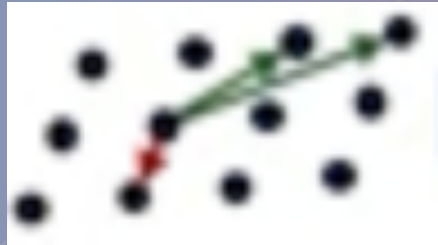
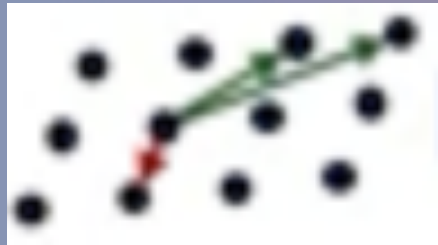
Signature

- Dilithium 
- Falcon 
- SPHINCS+ 

Key Encapsulation

- Kyber 

Signature

- Dilithium 
- Falcon 
- SPHINCS+ 



Post-Quantum transaction signatures

Desiderata



01 Secure

- Post-quantum secure
- Strong classical security
- Minimal assumptions



02 Efficient

- Small signatures
- Fast to sign and verify
- Low bandwidth and storage



03 Integrable

- Simple to implement
- Low complexity
- Fits existing formats



04 Lightweight

- Low memory footprint
- Low CPU and energy
- No heavy hardware



05 Deployable

- No trusted setup
- Compatible with validators
- Graceful upgrade path



06 Robust

- Long-term security margin
- Reasonable parameters
- Sustainable for decades



NIST PQ Signature Schemes

Scheme	Signature Size (bytes)	Public Key Size (bytes)	Gas (no prec.)	Aggregat.	HW Impl.	TSS Ease	PK Recovery
Falcon-512	666	897	Yes	Yes	Signing	Naive	Yes
Dilithium	2420	1312	No	No	Yes	Yes	No
SPHINCS+	7856	32	No	No	Key Generation	No	No



Good



Partial / Trade-off



Limited / Not supported



NIST PQ Signature Schemes

Scheme	Signature Size (bytes)	Public Key Size (bytes)	Gas (no prec.)	Aggregat.	HW Impl.	TSS Ease	PK Recovery
Falcon-512	666	897	Yes	Yes	Signing	Naive	Yes
Dilithium	2420	1312	No	No	Yes	Yes	No
SPHINCS+	7856	32	No	No	Key Generation	No	No



Good



Partial / Trade-off



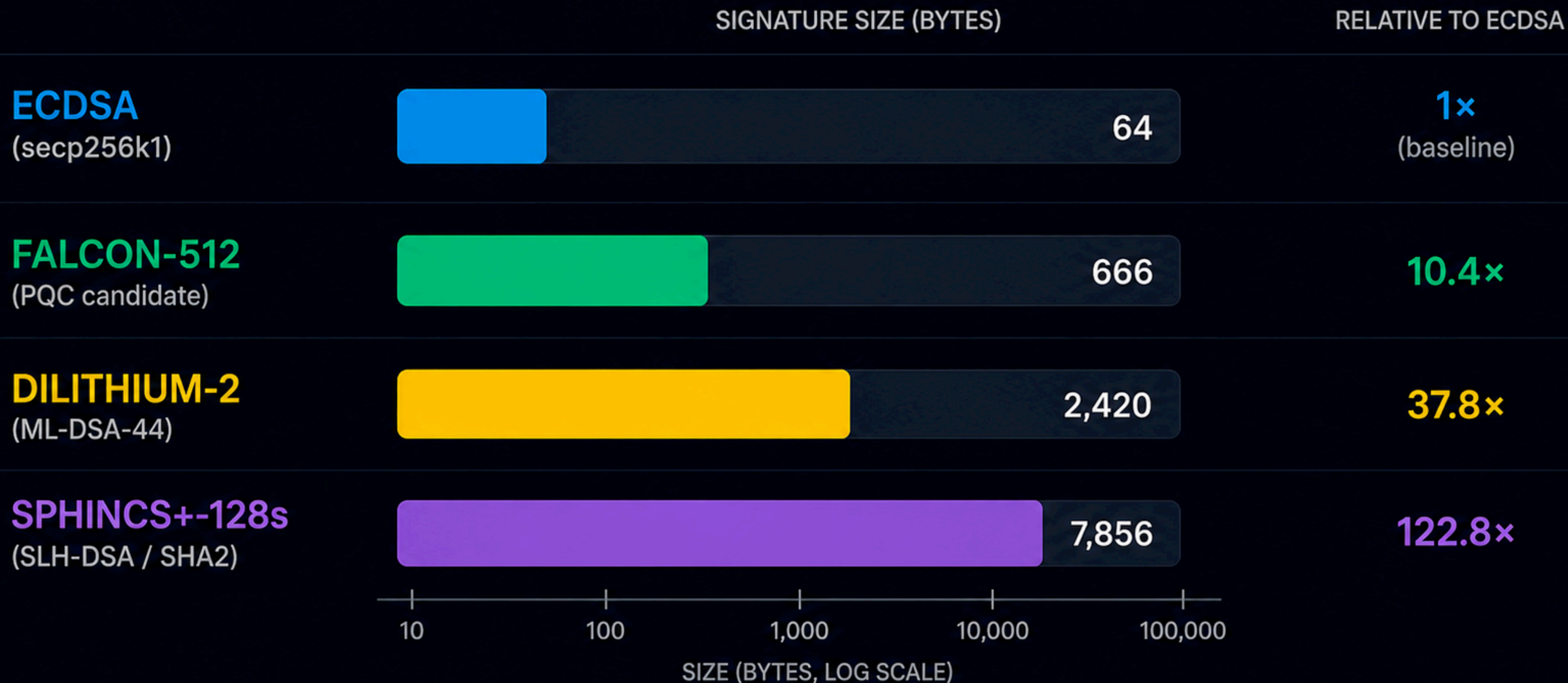
Limited / Not supported

No row is all green
(Pick your poison)



Signature Size Comparison

Smaller is better





NIST PQ Signature Schemes

Scheme	Signature Size (bytes)	Public Key Size (bytes)	Gas (no prec.)	Aggregat.	HW Impl.	TSS Ease	PK Recovery
Falcon-512	666	897	Yes	Yes	Signing	Naive	Yes
Dilithium	2420	1312	No	No	Yes	Yes	No
SPHINCS+	7856	32	No	No	Key Generation	No	No

 Good

 Partial / Trade-off

 Limited / Not supported

What really matters is not signature size but [signature aggregation](#) at the [P2P](#) and [mempool](#) layers

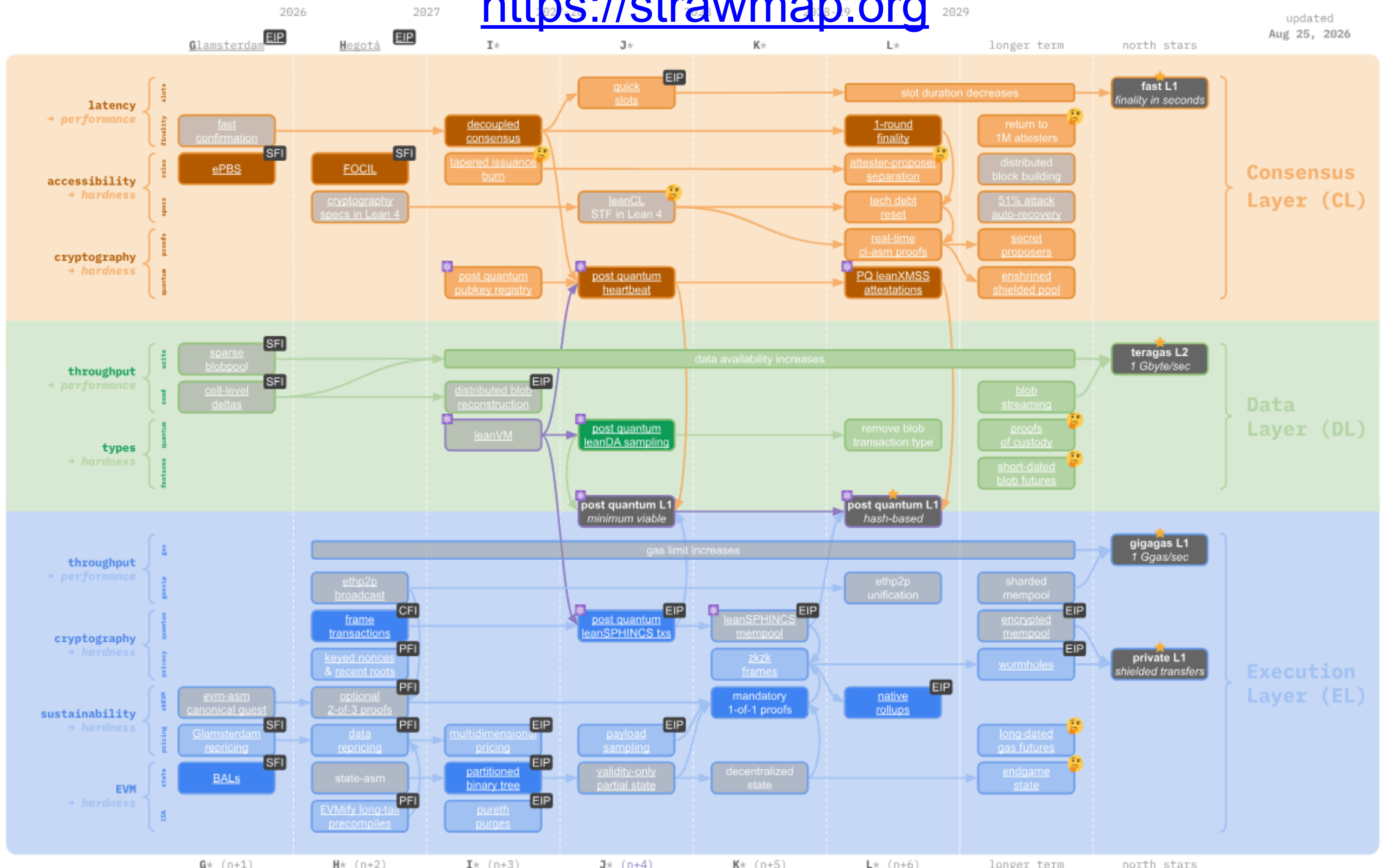


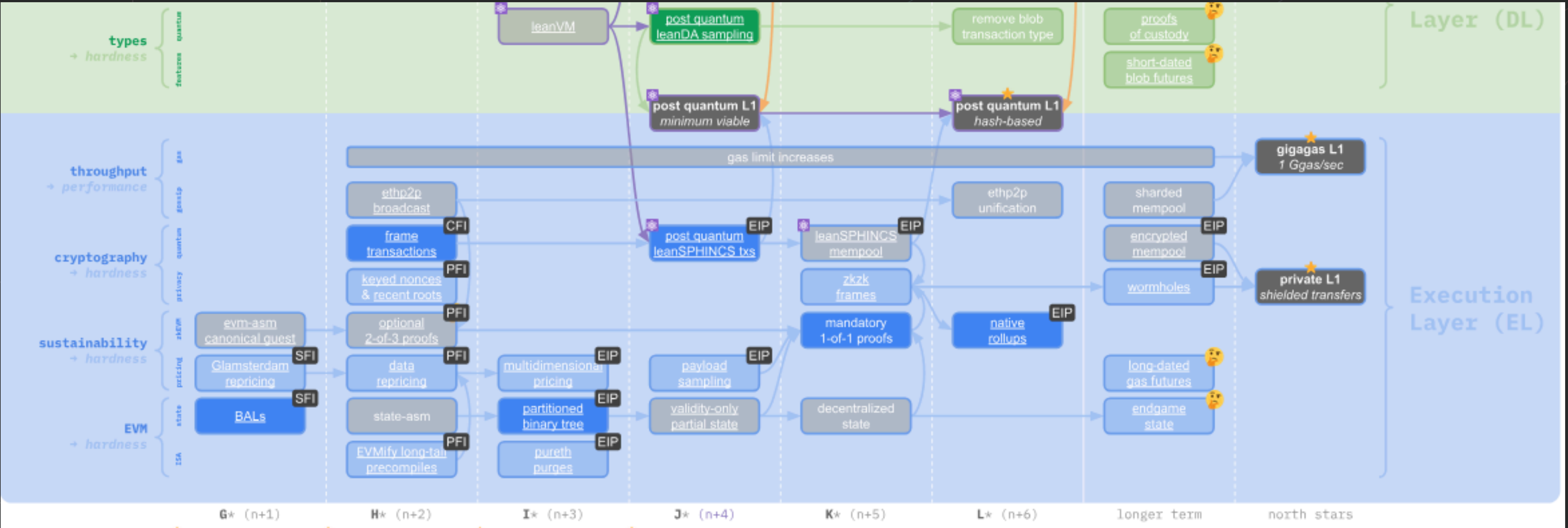
NIST PQ Signature Schemes

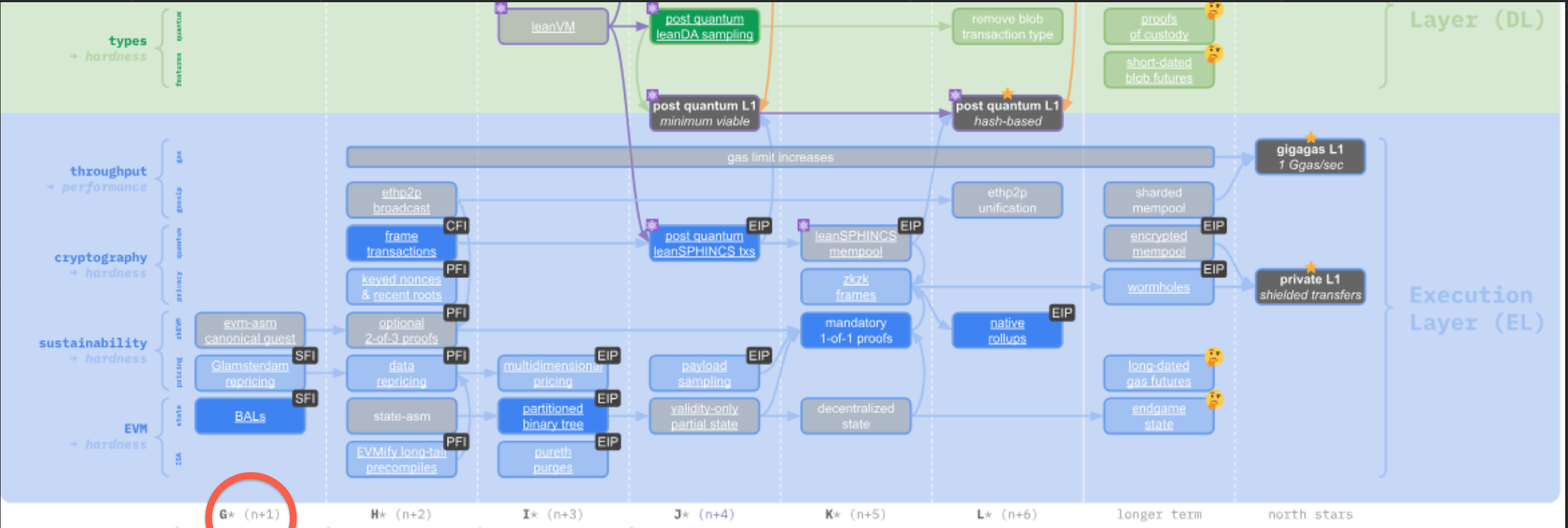
Scheme	Signature Size (bytes)	Public Key Size (bytes)	Gas (no prec.)	Aggregat.	HW Impl.	TSS Ease	PK Recovery
Falcon-512	666	897	Yes	Yes	Signing	Naive	Yes
Dilithium	2420	1312	No	No	Yes	Yes	No
SPHINCS+	7856	32	No	No	Key Generation	No	No

 Good  Partial / Trade-off  Limited / Not supported

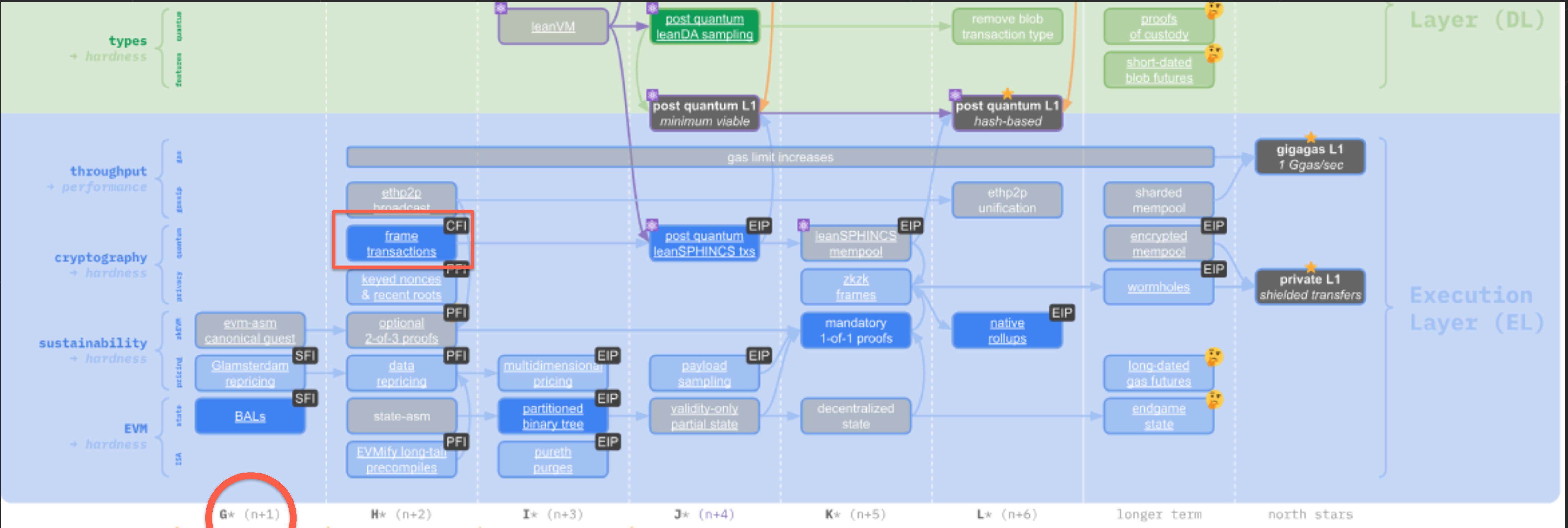
What really matters is not signature size but [signature aggregation](#) at the [P2P](#) and [mempool](#) layers



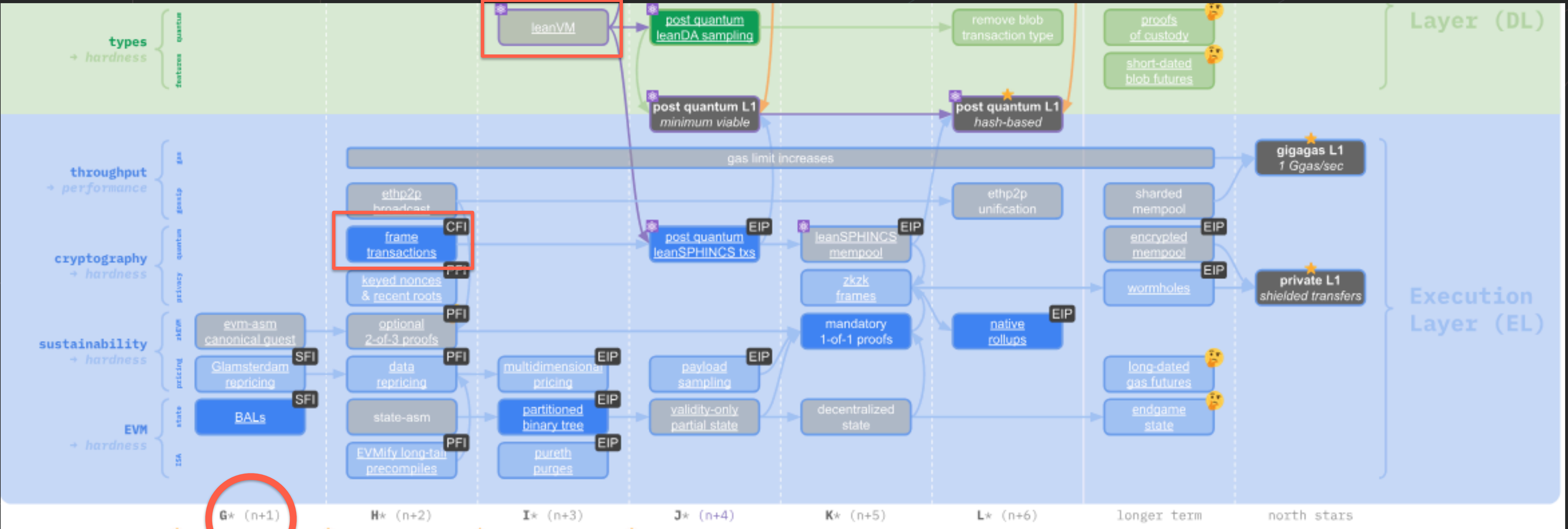




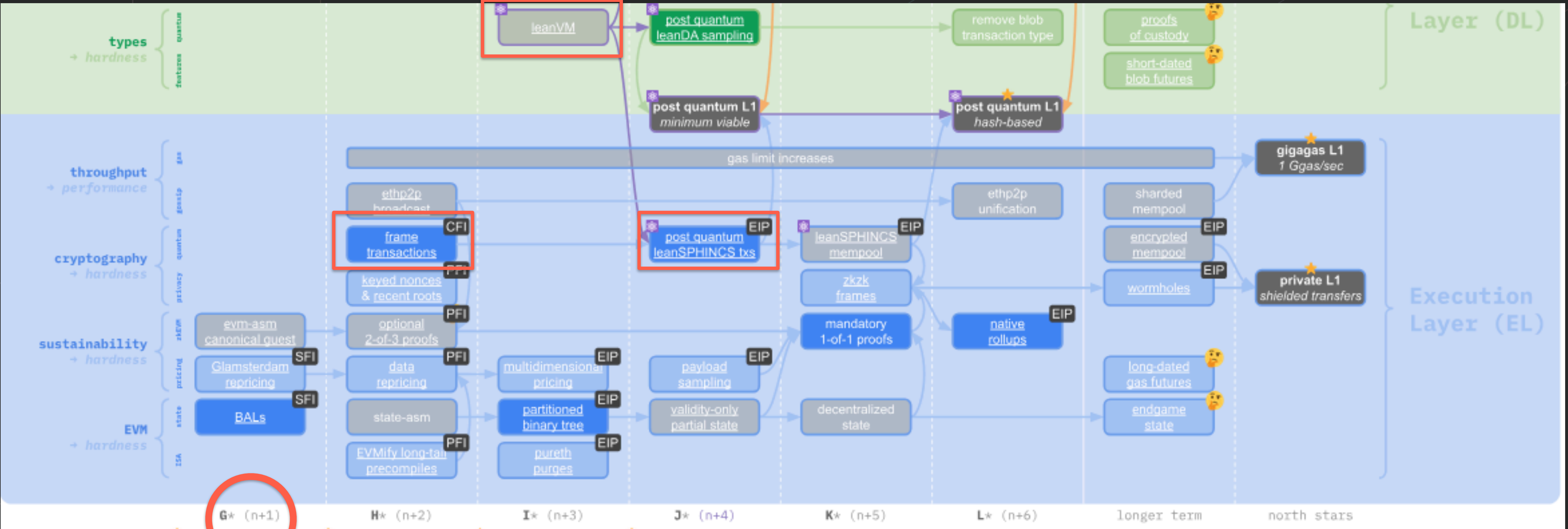
We are
here



We are
here



We are
here



We are
here



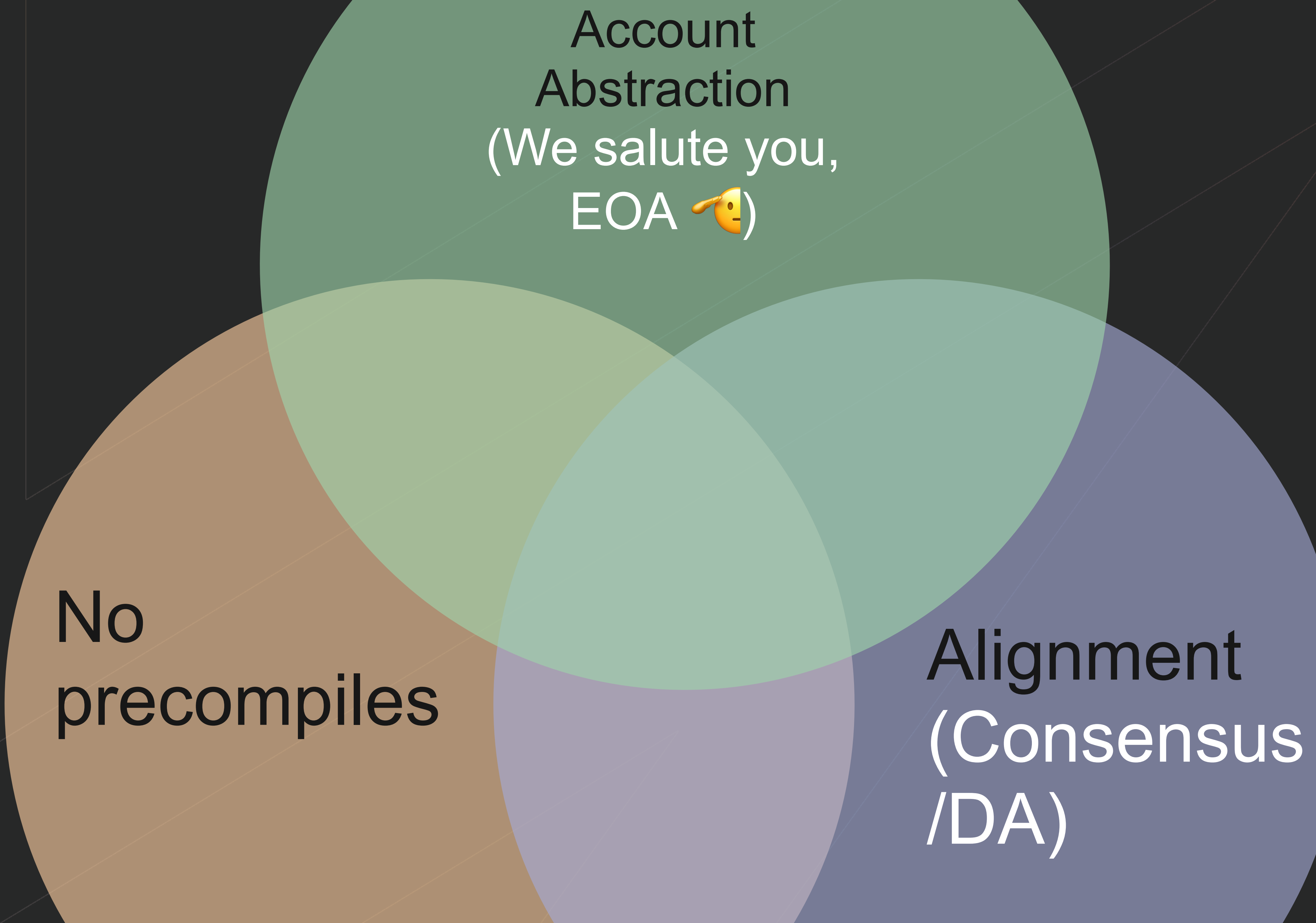
Account
Abstraction
(We salute you,
EOA 🧐)

A Venn diagram consisting of four overlapping circles. The top circle is green and contains the text. The bottom-left circle is orange. The bottom-right circle is light blue. The circle on the far right is dark blue. The circles overlap in various combinations, creating different shades of their respective colors.



Account
Abstraction
(We salute you,
EOA 🙋)

No
precompiles



Account
Abstraction
(We salute you,
EOA 🙌)

No
precompiles

Alignment
(Consensus
/DA)

Account
Abstraction
(We salute you,
EOA 🙋)

No
precompiles

lean
SPHINCS

Alignment
(Consensus
/DA)

Which Hash Function?



**SNARK-FRIENDLY
HASH**



**HASH-FRIENDLY
SNARK**

Which Hash Function?



Goodbye, Poseidon!

An epic 8-year, 8-figure rabbit hole in post-quantum cryptography reaches its dream conclusion. The Ethereum Foundation is abandoning Poseidon for L1, pivoting to SHA or BLAKE. This milestone unlocks ultimate security for lean Ethereum and foreshadows a golden era of hash-based cryptography.

Since 2018, the Ethereum Foundation has invested in magic cryptographic bricks, so-called "SNARK-friendly hashes". In 2019, Poseidon was born. It held strong and became the dominant SNARK-friendly hash, securing billions via zkrollups and zkVMs.

In a stunning reversal, breakthrough SNARK designs show that SNARK-friendly hashes aren't necessary after all. Off-the-shelf traditional hash functions like SHA2 and BLAKE2s can now match Poseidon in a SNARK. In hindsight the key was not SNARK-friendly hashes, but hash-friendly SNARKs.

Aug 2026

Hash based signatures

Stateless

- SPHINCS+ 

Statefull

- XMSS 
- LMS 

Hash based signatures

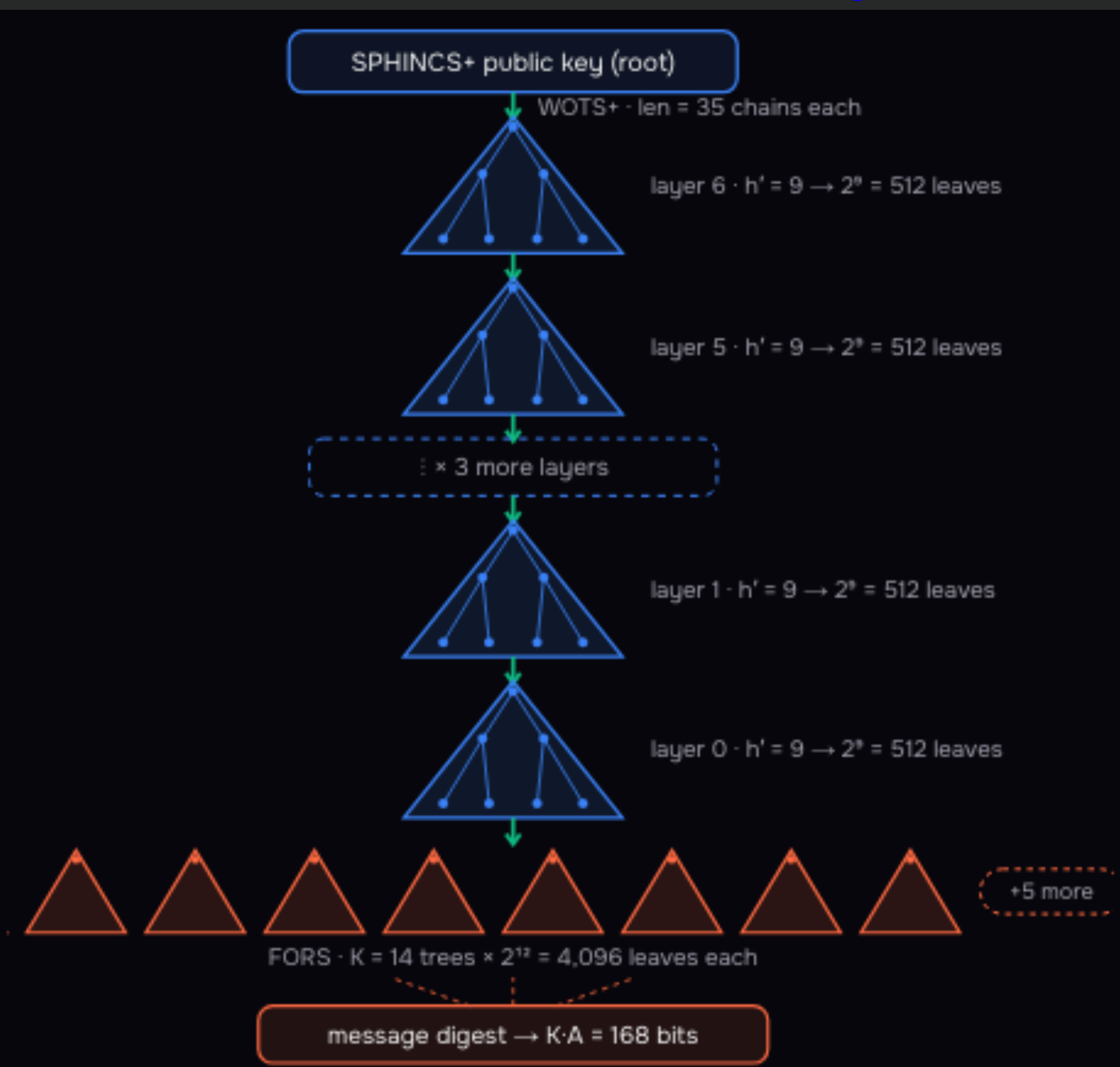
Stateless

- SPHINCS+ 



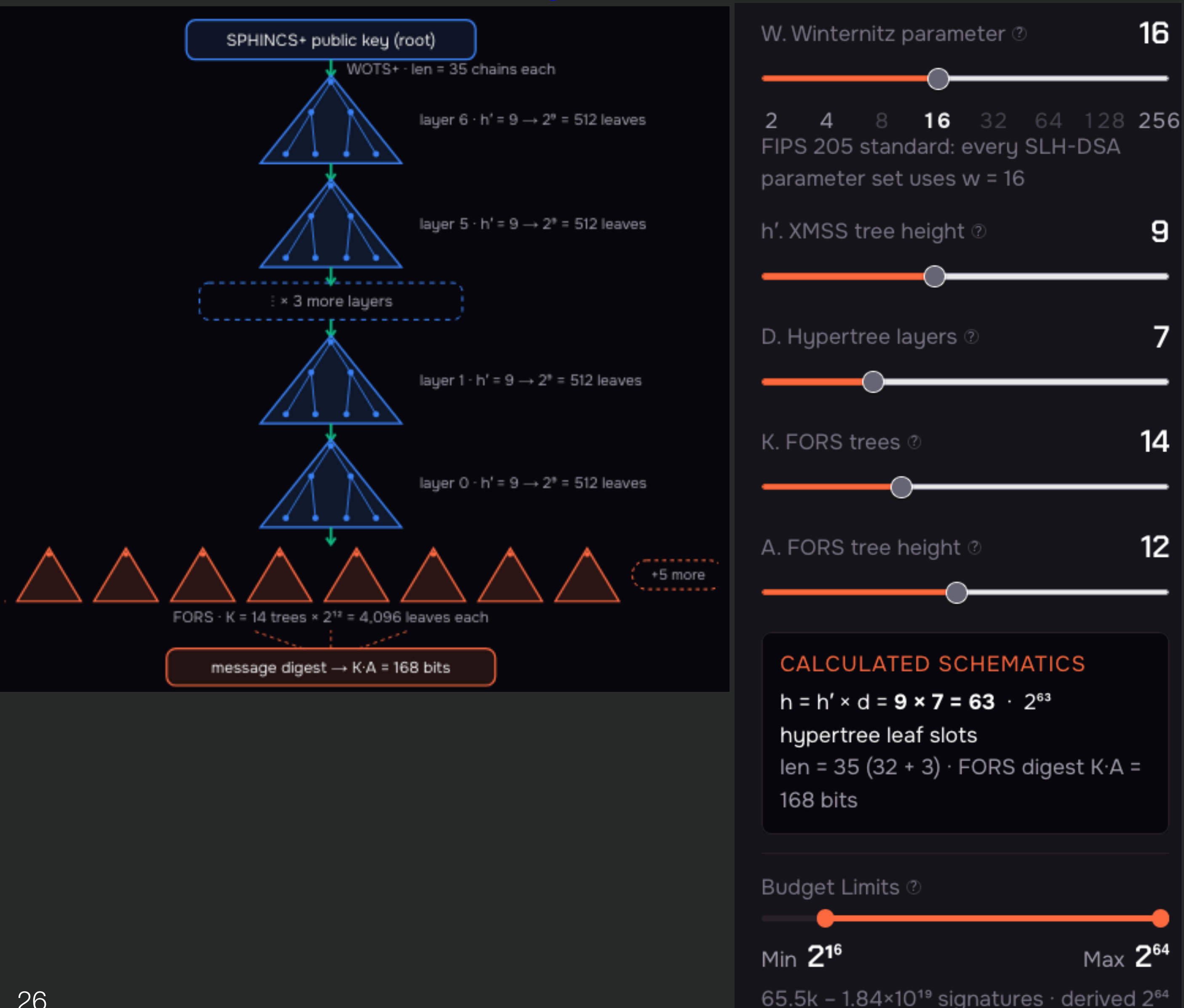
SLH-DSA-128s ★

<https://www.riva.xyz/tools/sphincs-parameter>



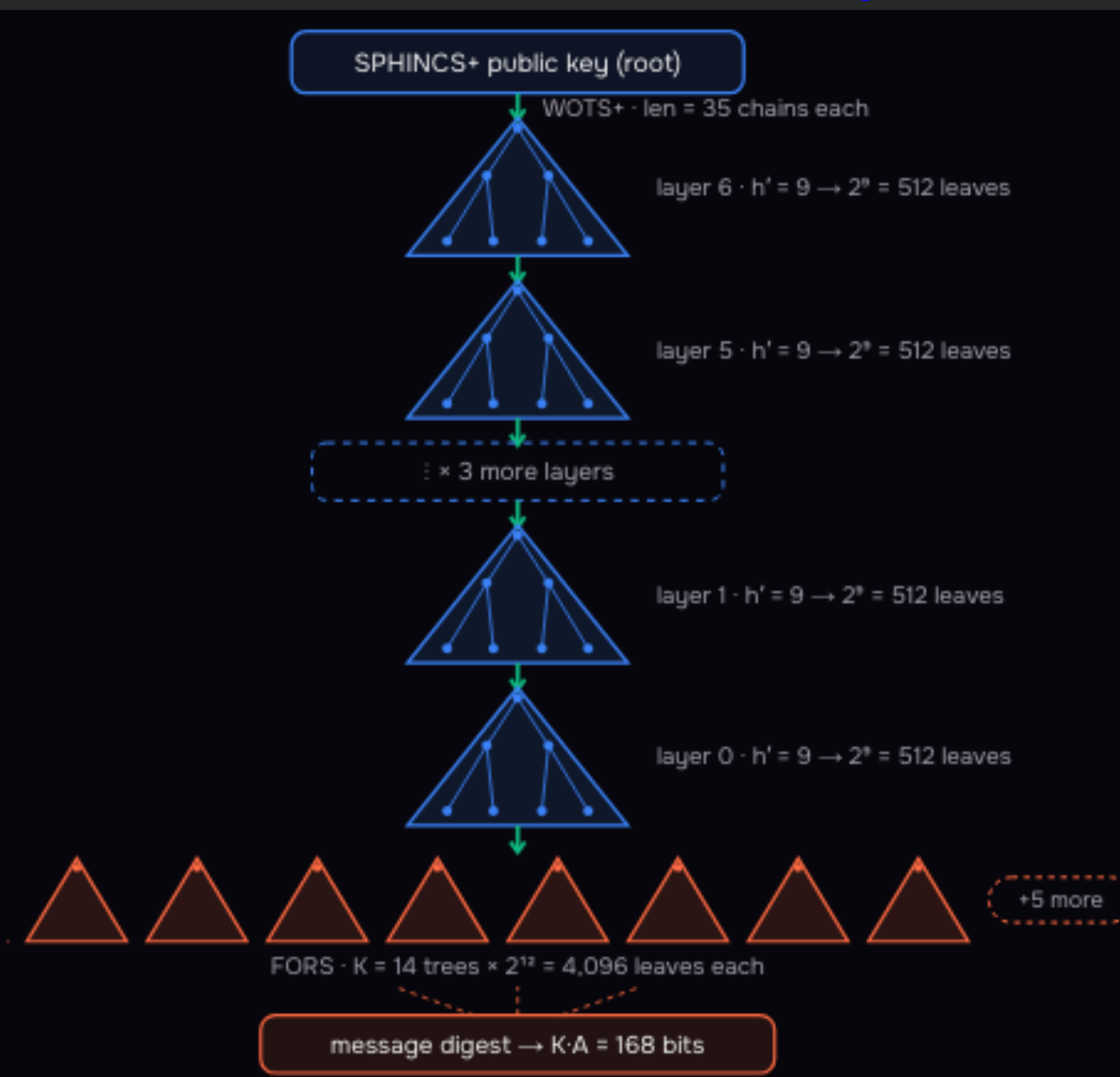
SLH-DSA-128s ★

<https://www.riva.xyz/tools/sphincs-parameter>



SLH-DSA-128s ★

<https://www.riva.xyz/tools/sphincs-parameter>



W. Winternitz parameter ⓘ **16**



2 4 8 **16** 32 64 128 256

FIPS 205 standard: every SLH-DSA parameter set uses w = 16

h'. XMSS tree height ⓘ **9**



D. Hypertree layers ⓘ **7**



K. FORS trees ⓘ **14**



A. FORS tree height ⓘ **12**



CALCULATED SCHEMATICS

$$h = h' \times d = 9 \times 7 = 63 \cdot 2^{63}$$

hypertree leaf slots

$$\text{len} = 35 (32 + 3) \cdot \text{FORS digest } K \cdot A = 168 \text{ bits}$$

Budget Limits ⓘ



Min **2¹⁶**

Max **2⁶⁴**

65.5k – 1.84×10¹⁹ signatures · derived 2⁶⁴

TOTAL SIGNATURE SIZE

7,856 B (7.7 KB)



● WOTS+ signatures 3.92 KB

● Hypertree auth 1.01 KB ● FORS 2.91 KB

● Randomizer 16 B

Verification gas (est.)

361.38k

calldata
125,696 +
3,928 hashes
× 60

Signing cost (hashes)

2.19M

≈ 219 ms at
10 MH/s

Verification hashes

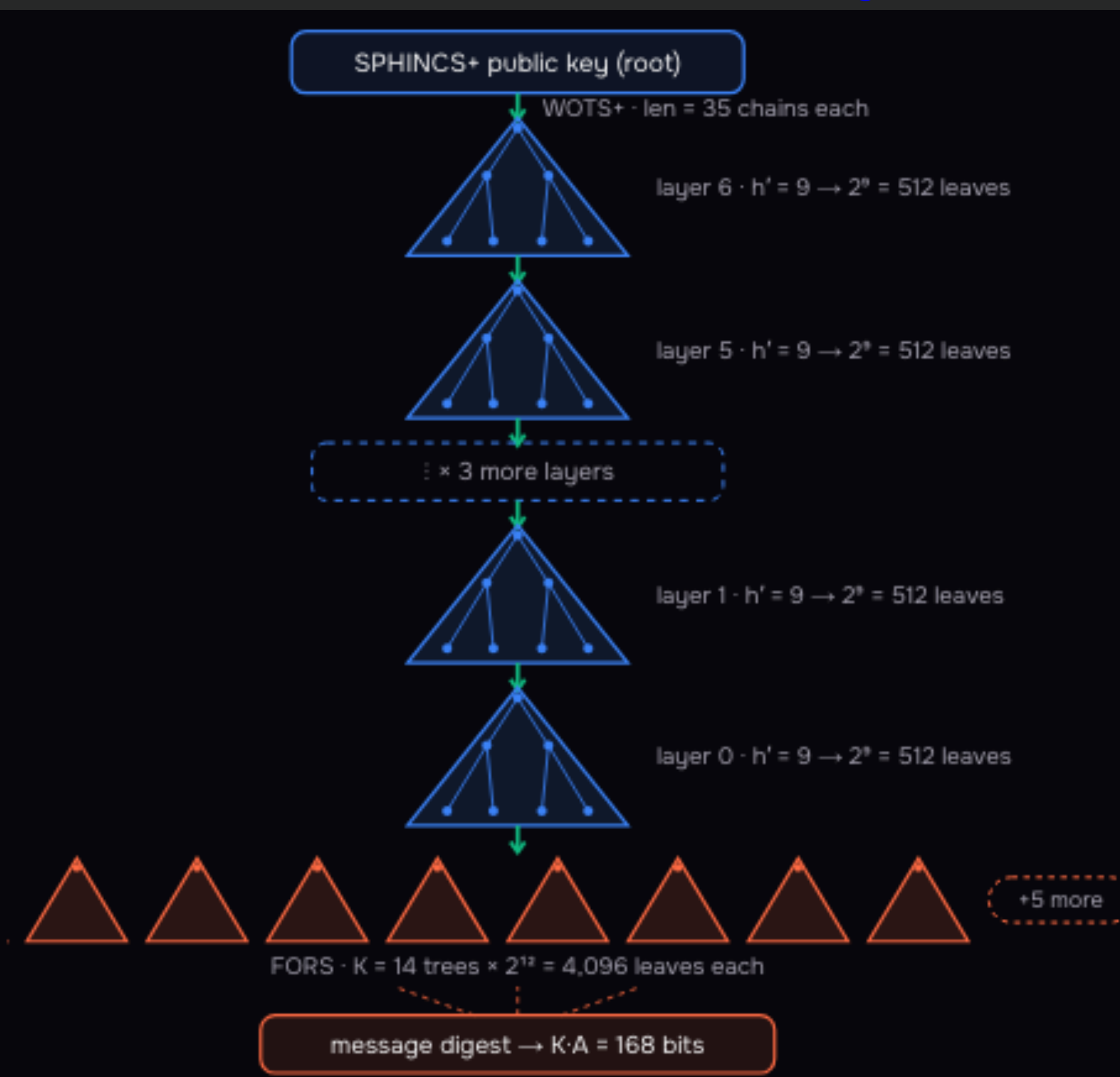
3,928

Public key

32 B

SLH-DSA-128s ★

<https://www.riva.xyz/tools/sphincs-parameter>



W. Winternitz parameter ⓘ **16**

2 4 8 **16** 32 64 128 256

FIPS 205 standard: every SLH-DSA parameter set uses $w = 16$

h' . XMSS tree height ⓘ **9**

D. Hypertree layers ⓘ **7**

K. FORS trees ⓘ **14**

A. FORS tree height ⓘ **12**

CALCULATED SCHEMATICS

$$h = h' \times d = 9 \times 7 = 63 \cdot 2^{63}$$

hypertree leaf slots

$$\text{len} = 35 (32 + 3) \cdot \text{FORS digest } K \cdot A = 168 \text{ bits}$$

Budget Limits ⓘ

Min **2^{16}**

Max **2^{64}**

65.5k – 1.84×10^{19} signatures · derived 2^{64}



TOTAL SIGNATURE SIZE

7,856 B (7.7 KB)

● WOTS+ signatures 3.92 KB

● Hypertree auth 1.01 KB ● FORS 2.91 KB

● Randomizer 16 B

Verification gas (est.)

361.38k

calldata
125,696 +
3,928 hashes
× 60

Signing cost (hashes)

2.19M

≈ 219 ms at
10 MH/s

Verification hashes

3,928

Public key

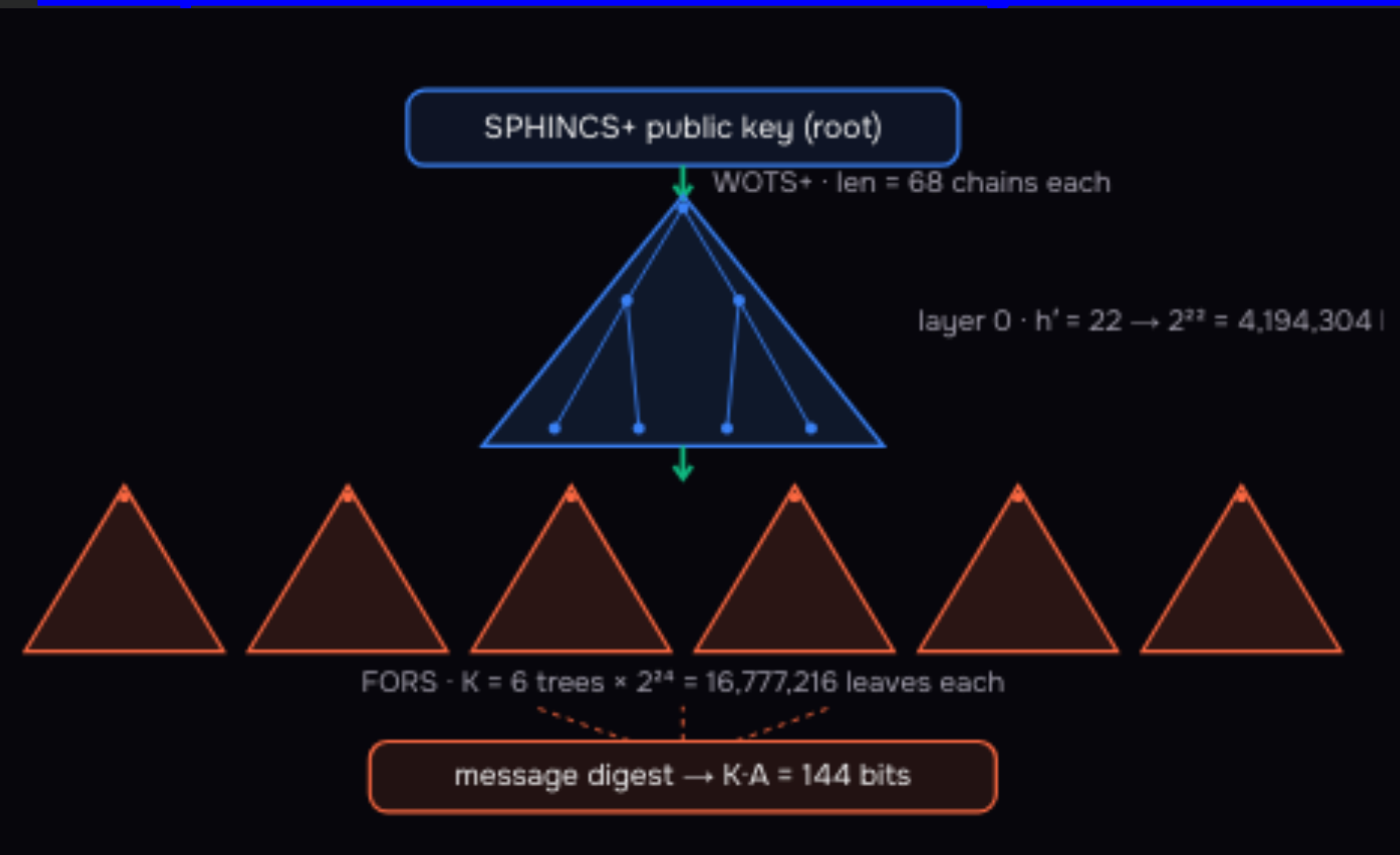
32 B

Additional SLH-DSA Parameter (April '26)

<https://www.riva.xyz/tools/sphincs-parameter>

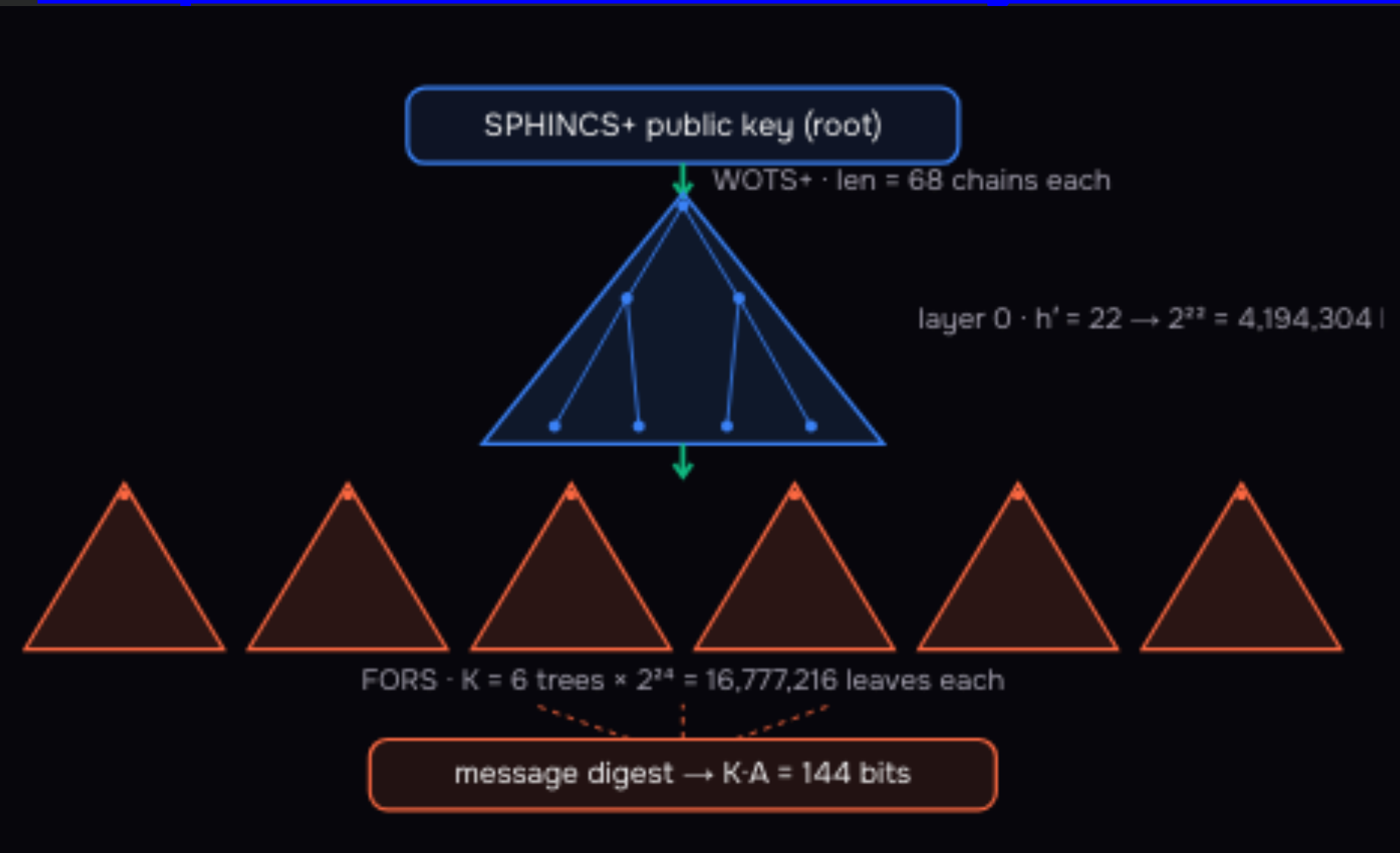
Additional SLH-DSA Parameter (April '26)

<https://www.riva.xyz/tools/sphincs-parameter>



Additional SLH-DSA Parameter (April '26)

<https://www.riva.xyz/tools/sphincs-parameter>



W. Winternitz parameter ⓘ **4**

2 4 8 **16** 32 64 128 256
byte-aligned ($\log_2 w$ divides 8), in the
SPHINCS+ submission's considered set

h'. XMSS tree height ⓘ **22**

D. Hypertree layers ⓘ **1**

K. FORS trees ⓘ **6**

A. FORS tree height ⓘ **24**

CALCULATED SCHEMATICS

$$h = h' \times d = 22 \times 1 = 22 \cdot 2^{22}$$

hypertree leaf slots

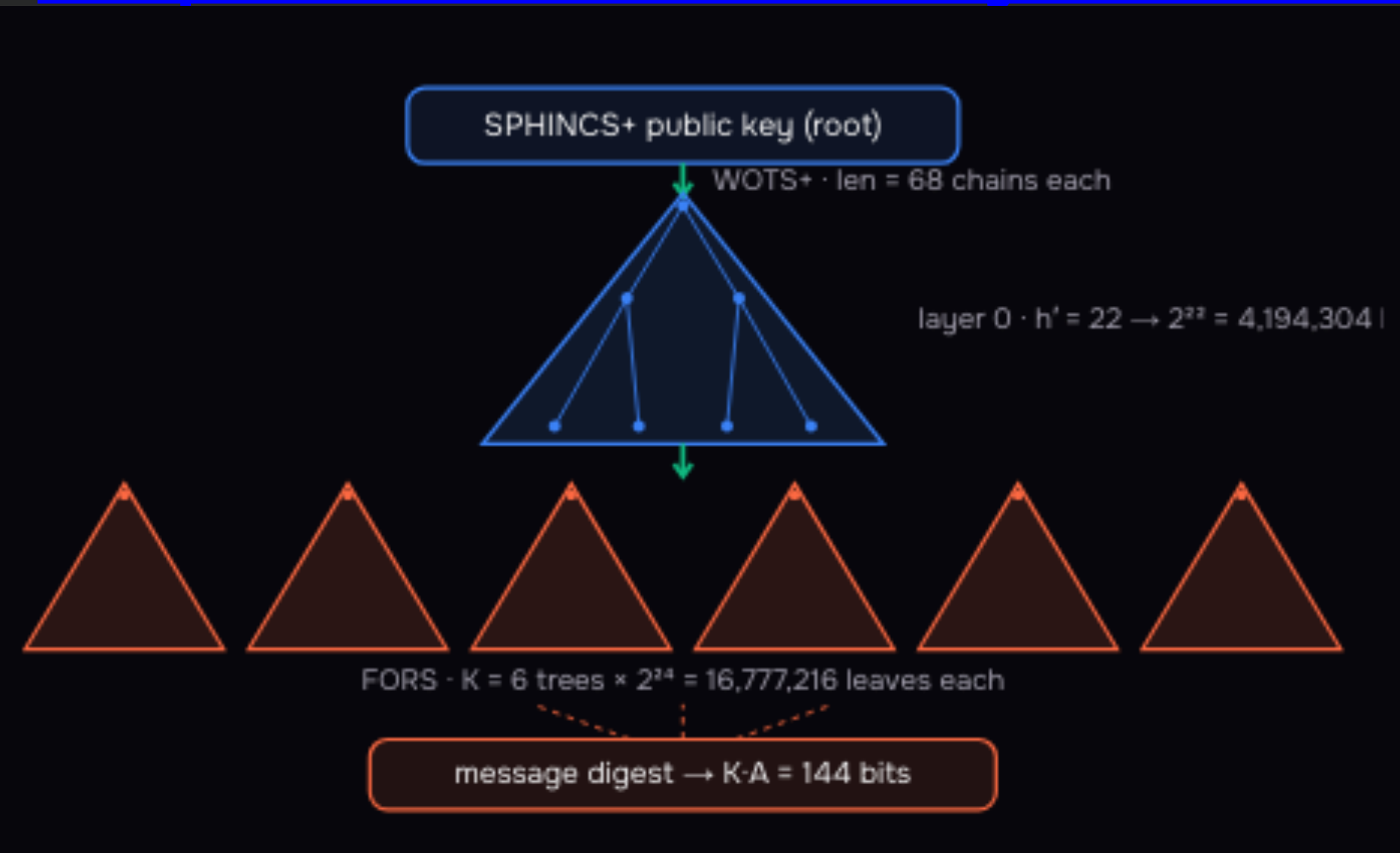
$$\text{len} = 68 (64 + 4) \cdot \text{FORS digest } K \cdot A = 144 \text{ bits}$$

Budget Limits ⓘ

Min **2¹⁶** Max **2²⁴**

Additional SLH-DSA Parameter (April '26)

<https://www.riva.xyz/tools/sphincs-parameter>



W. Winternitz parameter ⓘ **4**

2 4 8 **16** 32 64 128 256
byte-aligned ($\log_2 w$ divides 8), in the SPHINCS+ submission's considered set

h'. XMSS tree height ⓘ **22**

D. Hypertree layers ⓘ **1**

K. FORS trees ⓘ **6**

A. FORS tree height ⓘ **24**

CALCULATED SCHEMATICS
 $h = h' \times d = 22 \times 1 = 22 \cdot 2^{22}$
hypertree leaf slots
 $\text{len} = 68 (64 + 4) \cdot \text{FORS digest } K \cdot A = 144 \text{ bits}$

Budget Limits ⓘ

Min **2¹⁶** Max **2²⁴**

TOTAL SIGNATURE SIZE

3,856 B (3.8 KB)

- WOTS+ signatures 1.09 KB
- Hypertree auth 352 B
- FORS 2.40 KB
- Randomizer 16 B

Verification gas (est.)

84.38k

calldata
61,696 + 378
hashes × 60

Signing cost (hashes)

1.45B

≈ 2.42 min at
10 MH/s

Verification hashes

378

worst case ·
avg 276

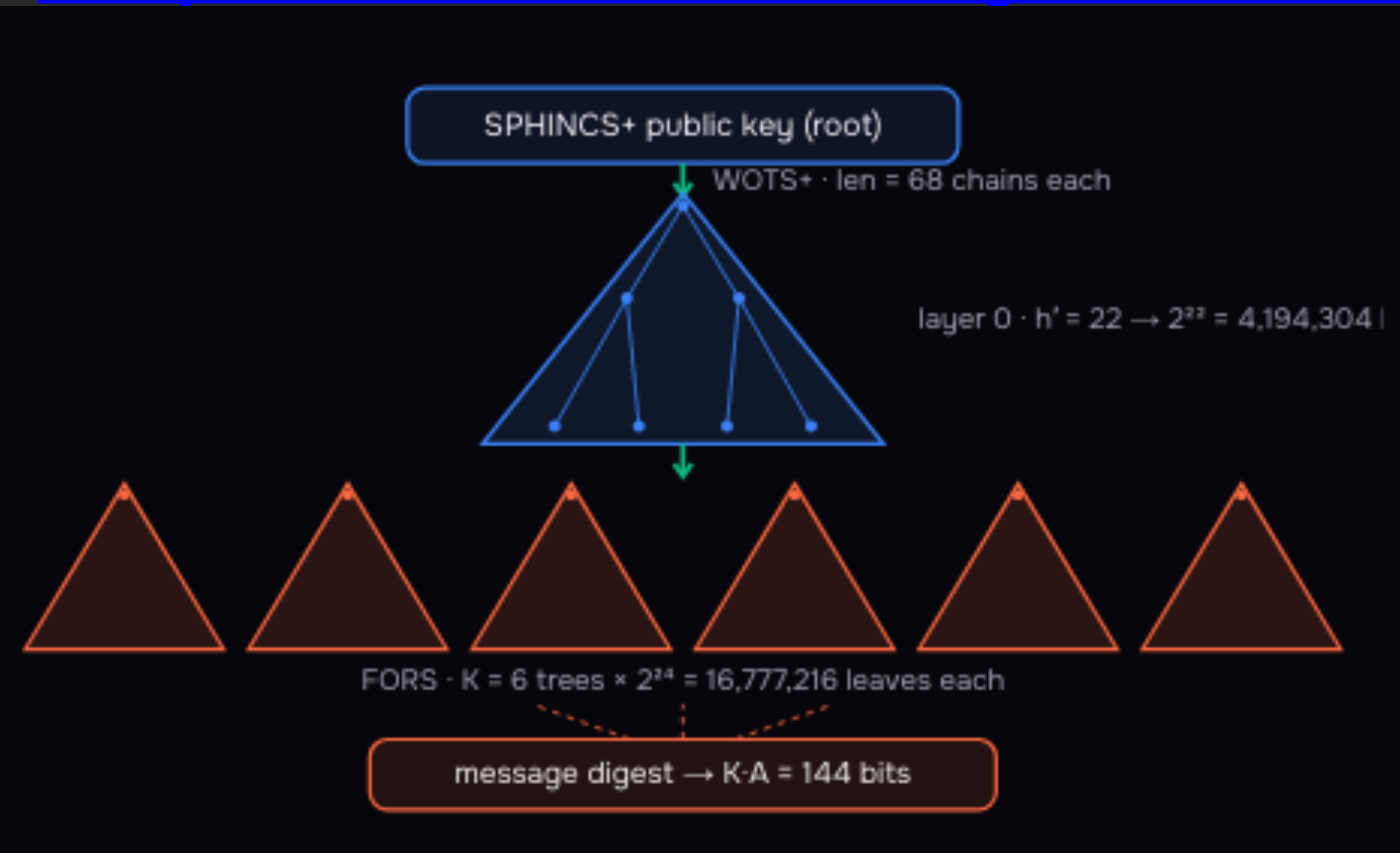
Public key

32 B

secret key 64
B

Additional SLH-DSA Parameter (April '26)

<https://www.riva.xyz/tools/sphincs-parameter>



W. Winternitz parameter ⓘ **4**

2 4 8 **16** 32 64 128 256
byte-aligned ($\log_2 w$ divides 8), in the SPHINCS+ submission's considered set

h' . XMSS tree height ⓘ **22**

D. Hypertree layers ⓘ **1**

K. FORS trees ⓘ **6**

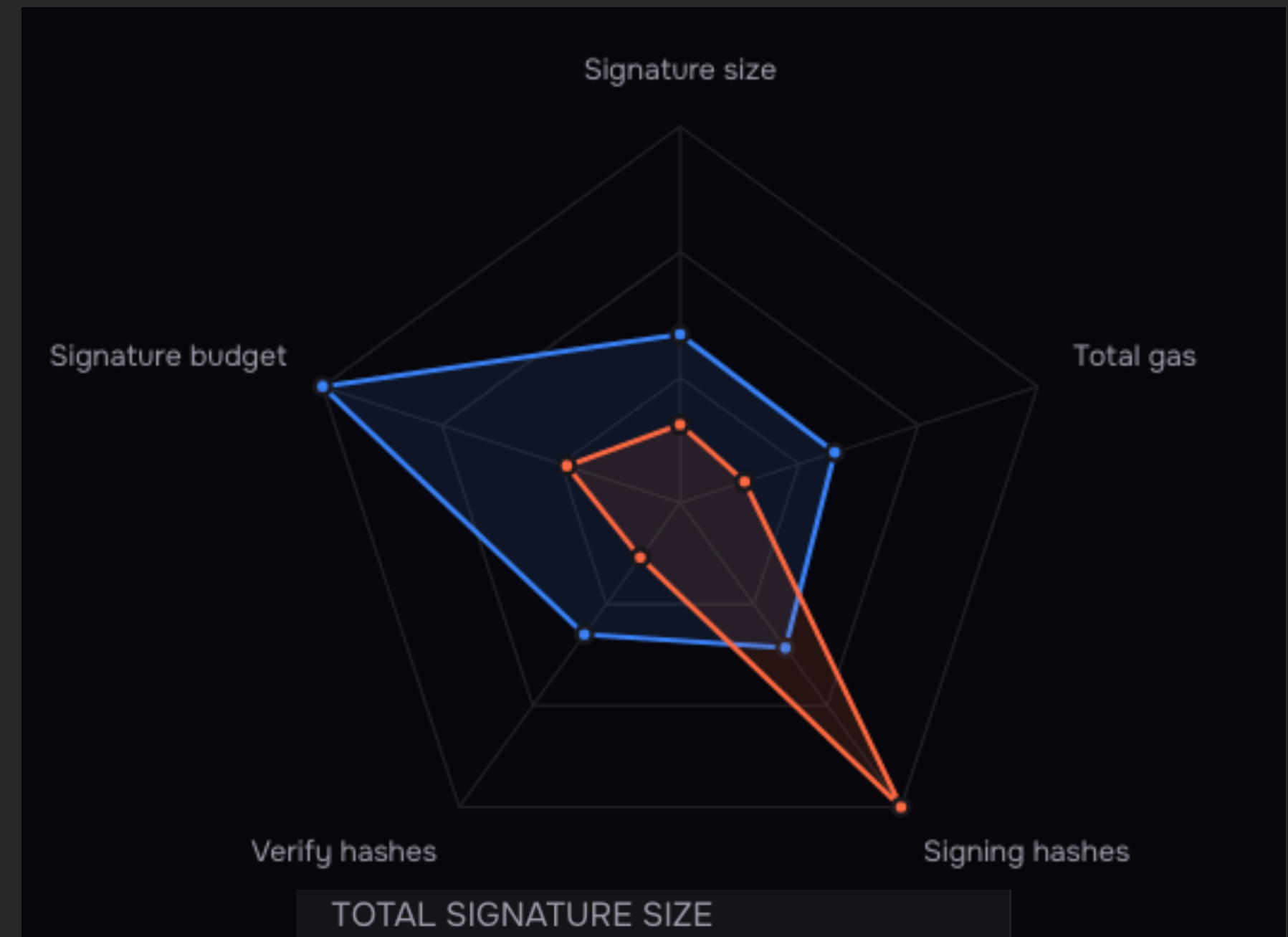
A. FORS tree height ⓘ **24**

CALCULATED SCHEMATICS

$h = h' \times d = 22 \times 1 = 22 \cdot 2^{22}$
hypertree leaf slots
 $\text{len} = 68 (64 + 4) \cdot \text{FORS digest } K \cdot A = 144 \text{ bits}$

Budget Limits ⓘ

Min **2^{16}** Max **2^{24}**



TOTAL SIGNATURE SIZE

3,856 B (3.8 KB)

- WOTS+ signatures 1.09 KB
- Hypertree auth 352 B
- FORS 2.40 KB
- Randomizer 16 B

Verification gas (est.)

84.38k

calldata
 $61,696 + 378$
hashes $\times 60$

Signing cost (hashes)

1.45B

≈ 2.42 min at
10 MH/s

Verification hashes

378

worst case ·
avg 276

Public key

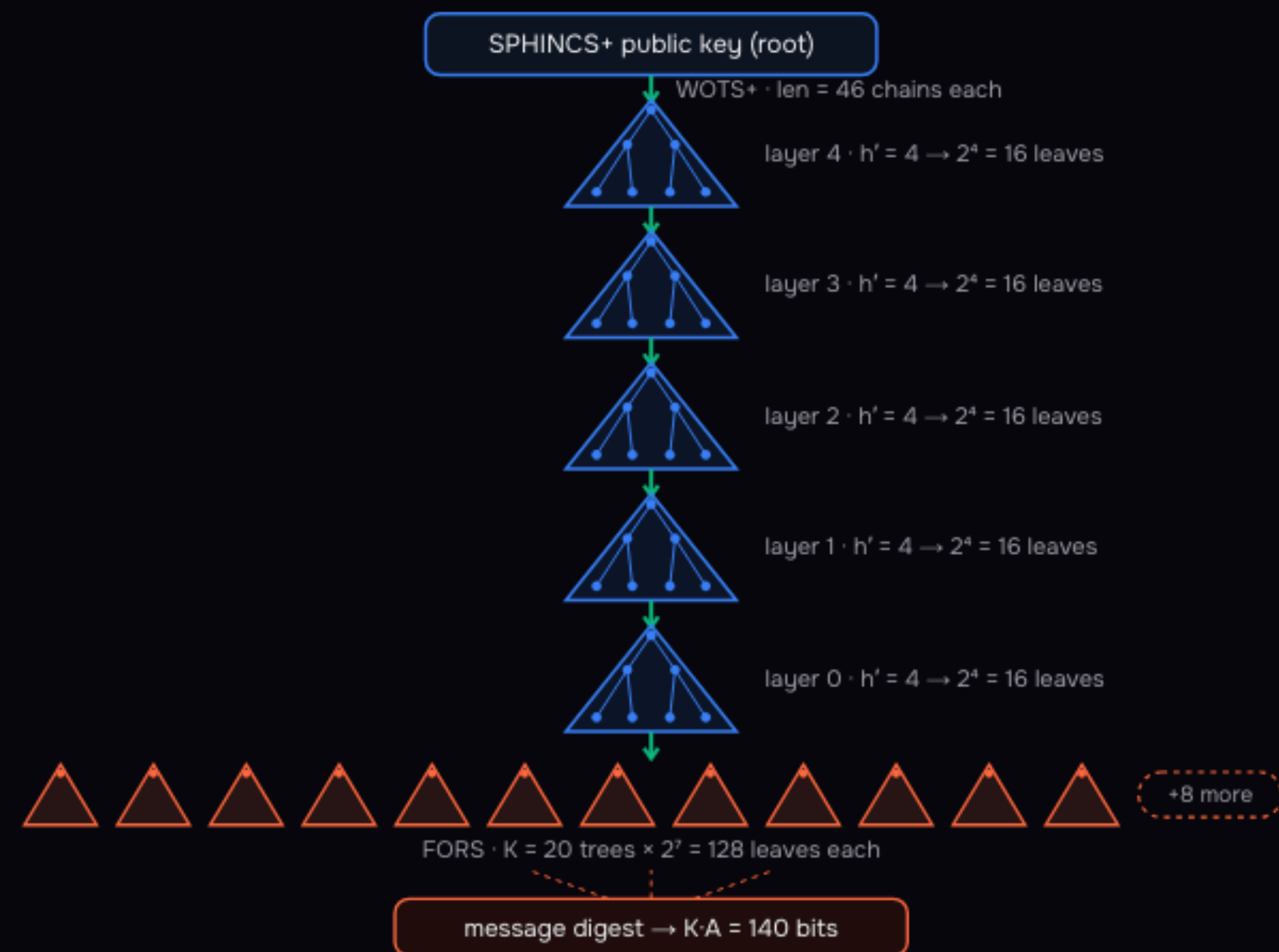
32 B

secret key 64 B

SPHINCS minus (C12) - Nico Consigny
<https://sphincsminus.org/>

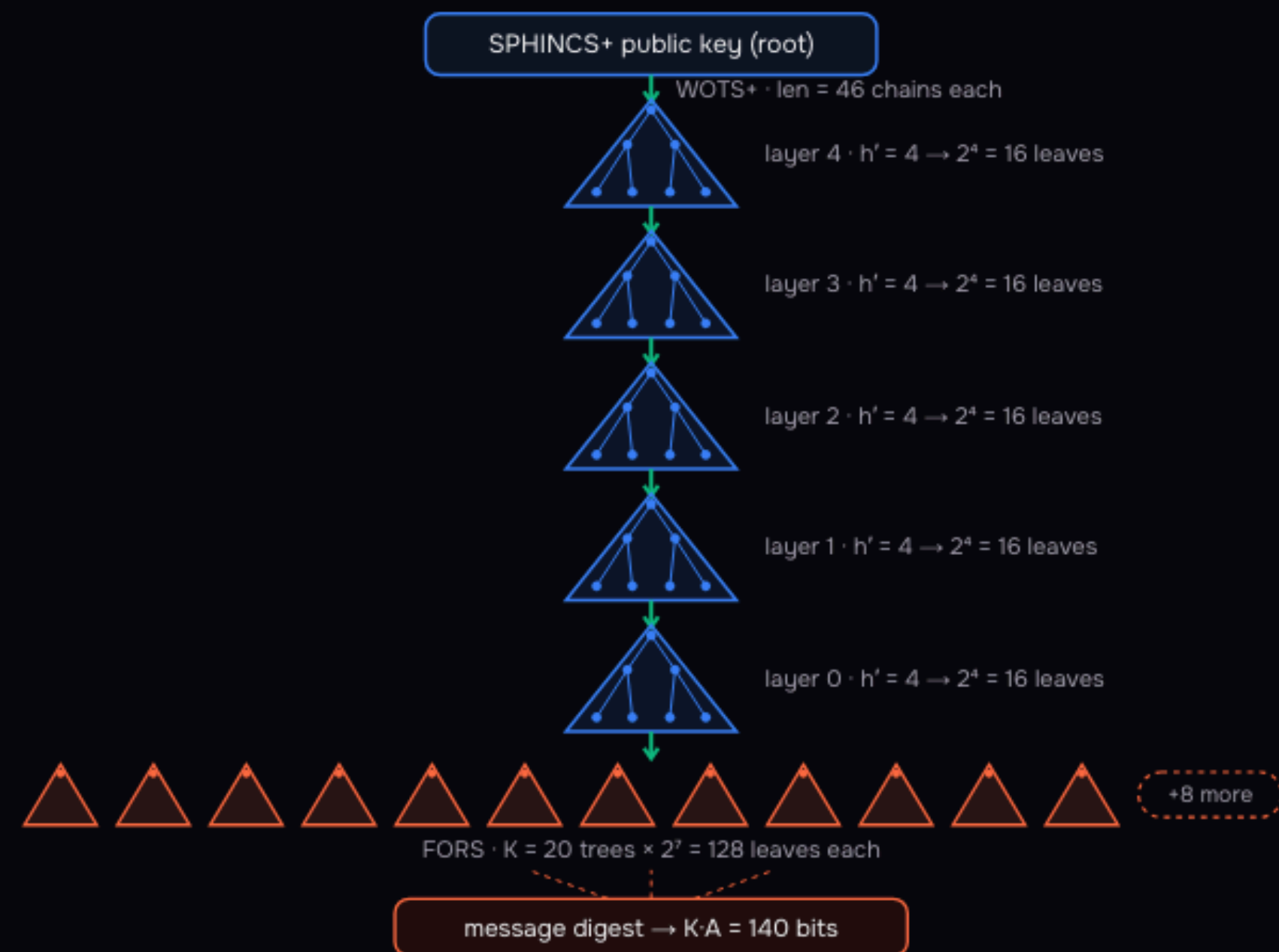
SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



W. Winternitz parameter ?

8



2 4 8 **16** 32 64 128 256

not byte-aligned: digits straddle byte boundaries, outside the standard set

h' . XMSS tree height ?

4



D. Hypertree layers ?

5



K. FORS trees ?

20



A. FORS tree height ?

7



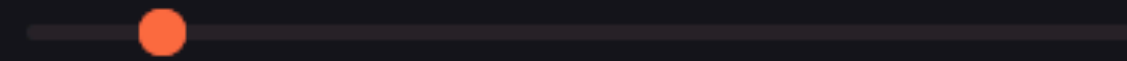
CALCULATED SCHEMATICS

$h = h' \times d = 4 \times 5 = 20 \cdot 2^{20}$ hypertree

leaf slots

len = 46 (43 + 3) · FORS digest $K \cdot A =$
140 bits

Budget Limits ?

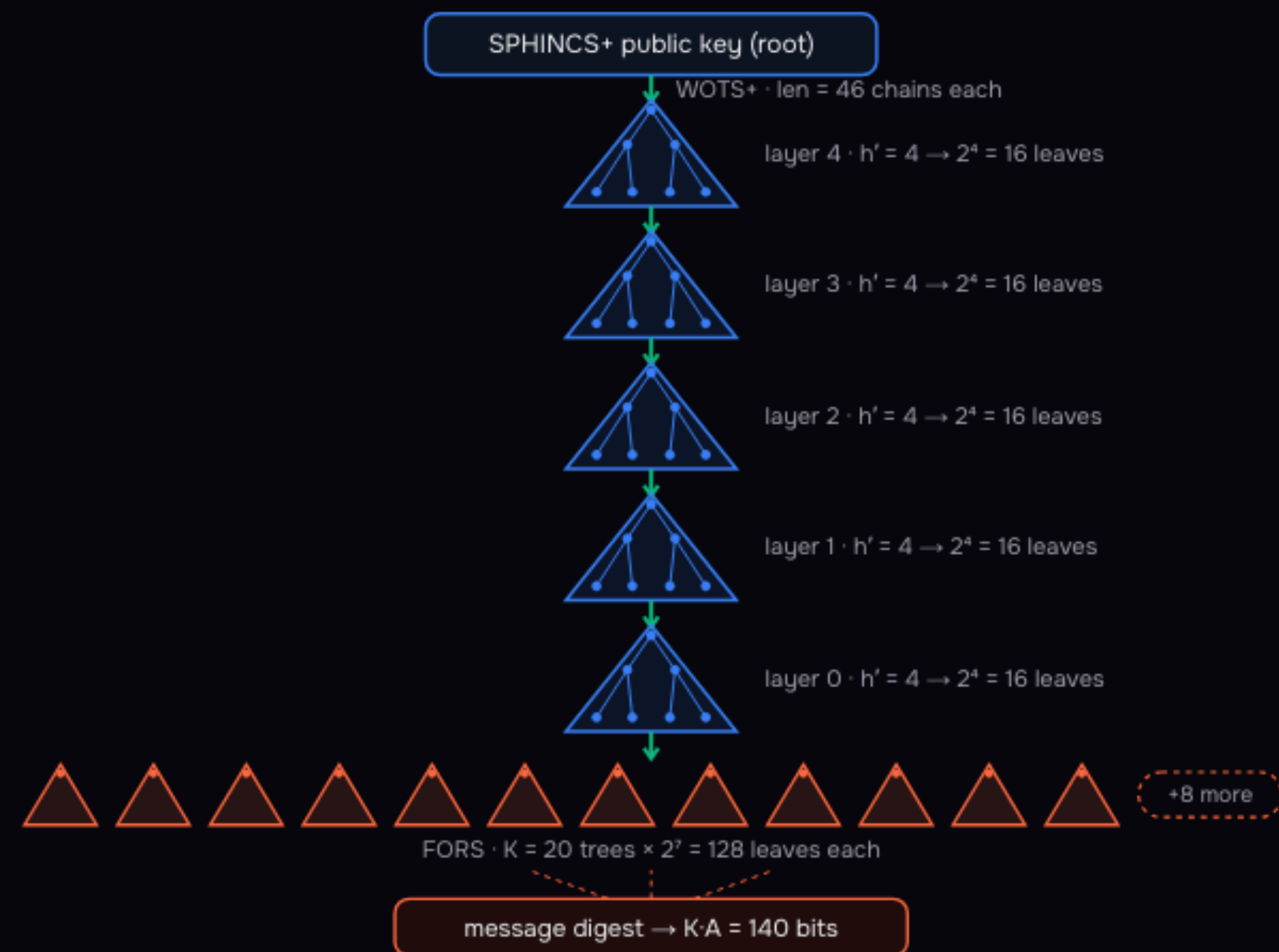


Min 2^{14}

Max 2^{14}

SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



Among Ethereum mainnet addresses that sent at least one transaction in 2025, **99.99%** sent fewer than **3,000** transactions that year.

W. Winternitz parameter ⓘ **8**

2 4 8 **16** 32 64 128 256
not byte-aligned: digits straddle byte boundaries, outside the standard set

h'. XMSS tree height ⓘ **4**

D. Hypertree layers ⓘ **5**

K. FORS trees ⓘ **20**

A. FORS tree height ⓘ **7**

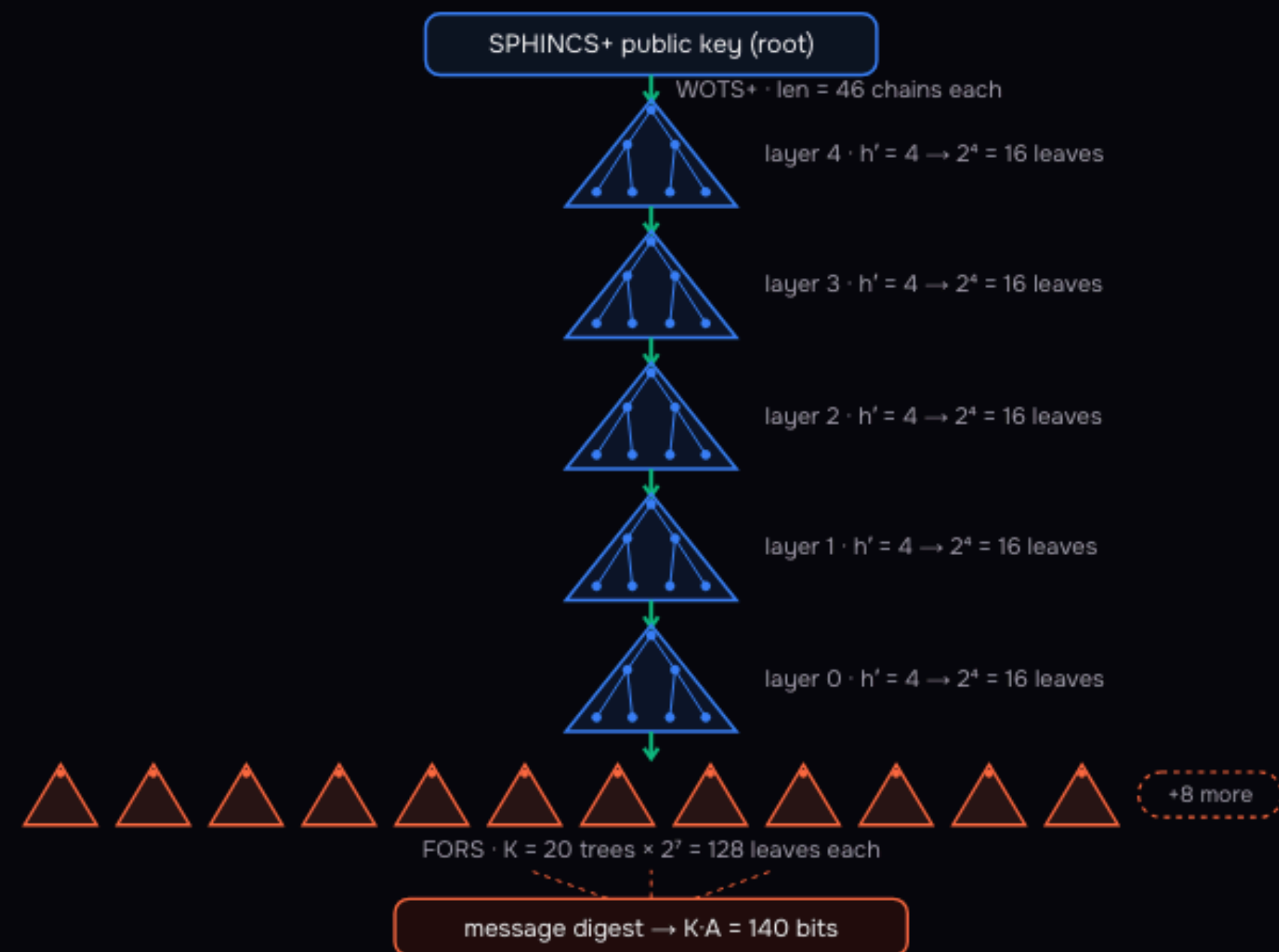
CALCULATED SCHEMATICS
 $h = h' \times d = 4 \times 5 = 20 \cdot 2^{20}$ hypertree leaf slots
 $len = 46 (43 + 3) \cdot \text{FORS digest } K \cdot A = 140 \text{ bits}$

Budget Limits ⓘ

Min **2¹⁴** Max **2¹⁴**

SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



W. Winternitz parameter ? 8

2 4 8 **16** 32 64 128 256
not byte-aligned: digits straddle byte boundaries, outside the standard set

h'. XMSS tree height ? 4

D. Hypertree layers ? 5

K. FORS trees ? 20

A. FORS tree height ? 7

CALCULATED SCHEMATICS

$h = h' \times d = 4 \times 5 = 20 \cdot 2^{20}$ hypertree
leaf slots
 $len = 46 (43 + 3) \cdot \text{FORS digest } K \cdot A = 140$ bits

Budget Limits ?

Min 2^{14}

Max 2^{14}

TOTAL SIGNATURE SIZE

6,576 B (6.4 KB)

- WOTS+ signatures 3.68 KB
- Hypertree auth 320 B
- FORS 2.56 KB
- Randomizer 16 B

Verification gas (est.)

212.98k

calldata
105,216 +
1,796 hashes
× 60

Signing cost (hashes)

37.26k

≈ 4 ms at 10
MH/s

Verification hashes

1,796

worst case ·

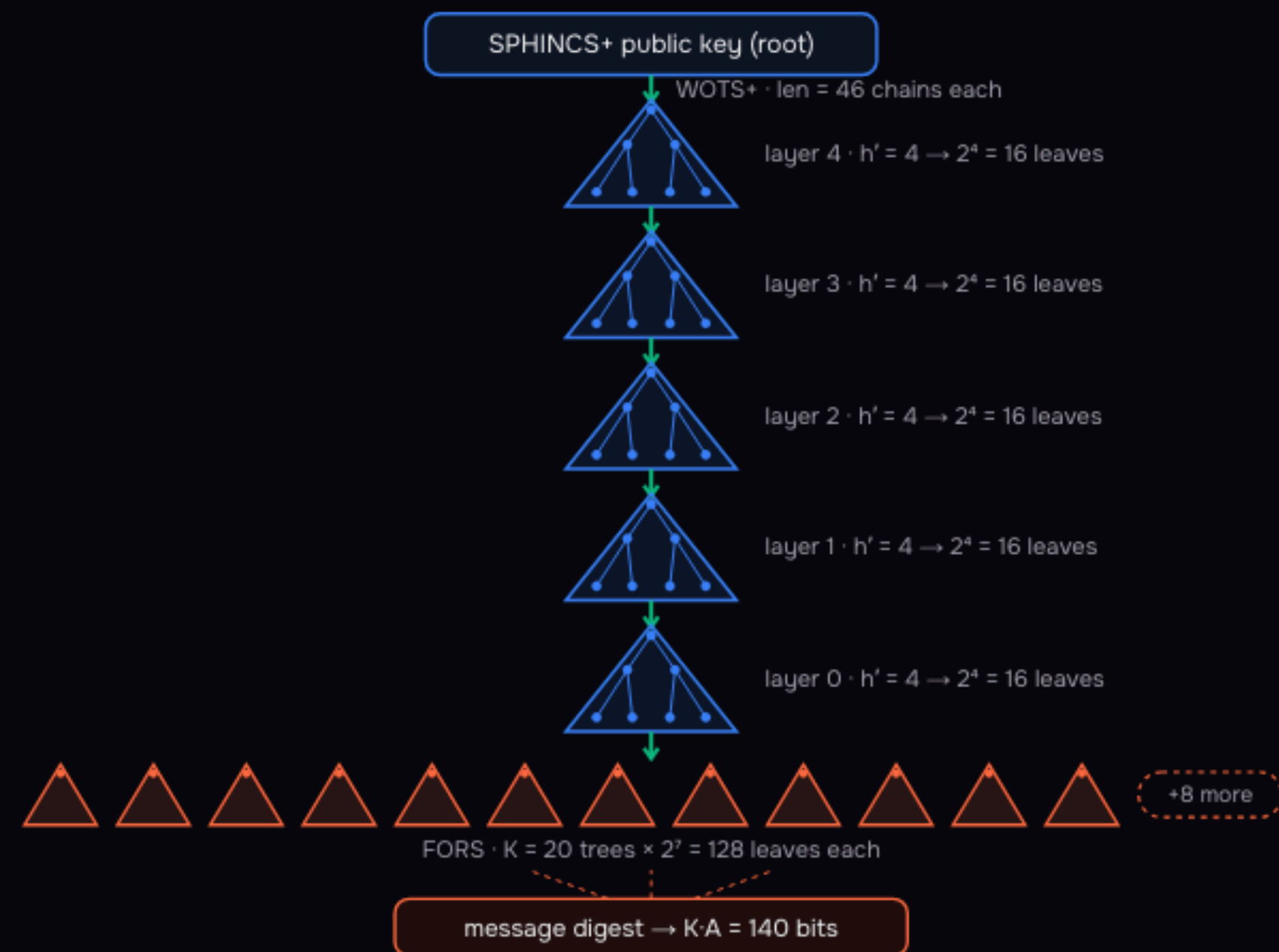
Public key

32 B

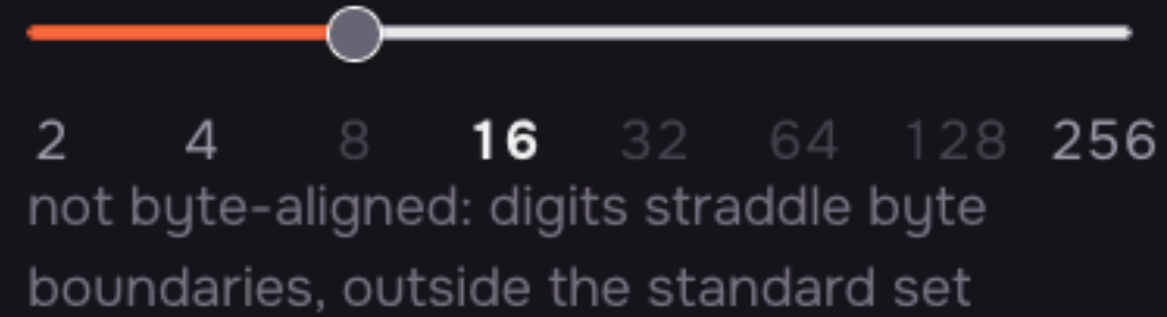
secret key 64
B

SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



W. Winternitz parameter ⓘ 8



h'. XMSS tree height ⓘ 4



D. Hypertree layers ⓘ 5



K. FORS trees ⓘ 20



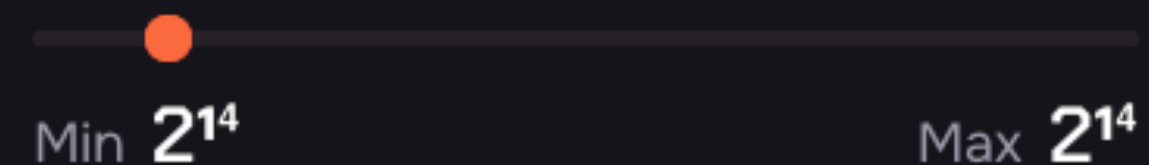
A. FORS tree height ⓘ 7



CALCULATED SCHEMATICS

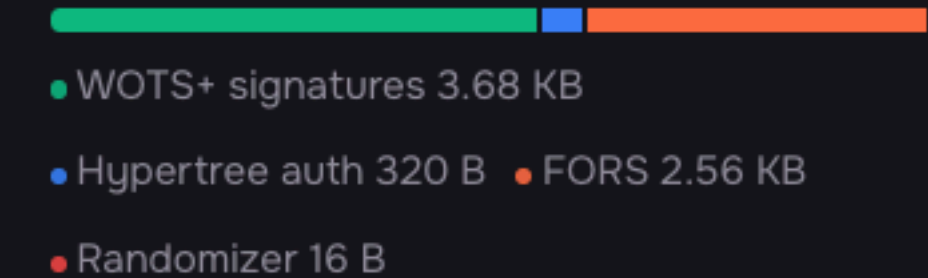
$h = h' \times d = 4 \times 5 = 20 \cdot 2^{20}$ hypertree leaf slots
 $\text{len} = 46 (43 + 3) \cdot \text{FORS digest } K \cdot A = 140 \text{ bits}$

Budget Limits ⓘ



TOTAL SIGNATURE SIZE

6,576 B (6.4 KB)



Verification gas (est.)

212.98k

calldata
105,216 +
1,796 hashes
× 60

Signing cost (hashes)

37.26k

≈ 4 ms at 10
MH/s

Verification hashes

1,796

worst case ·

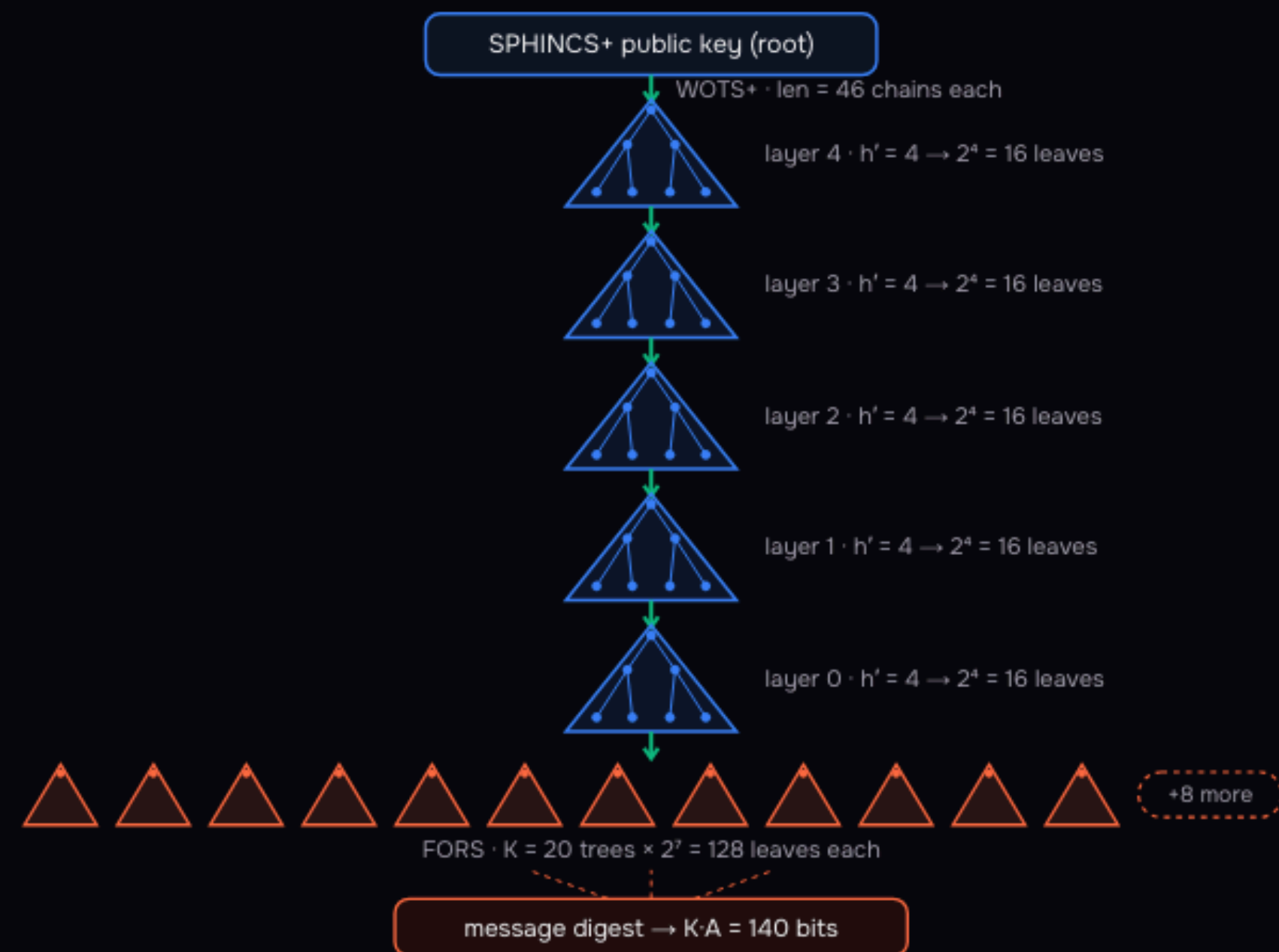
Public key

32 B

secret key 64
B

SPHINCS minus (C12) - Nico Consigny

<https://sphincsminus.org/>



W. Winternitz parameter ? 8

2 4 8 16 32 64 128 256
not byte-aligned: digits straddle byte boundaries, outside the standard set

h'. XMSS tree height ? 4

D. Hypertree layers ? 5

K. FORS trees ? 20

A. FORS tree height ? 7

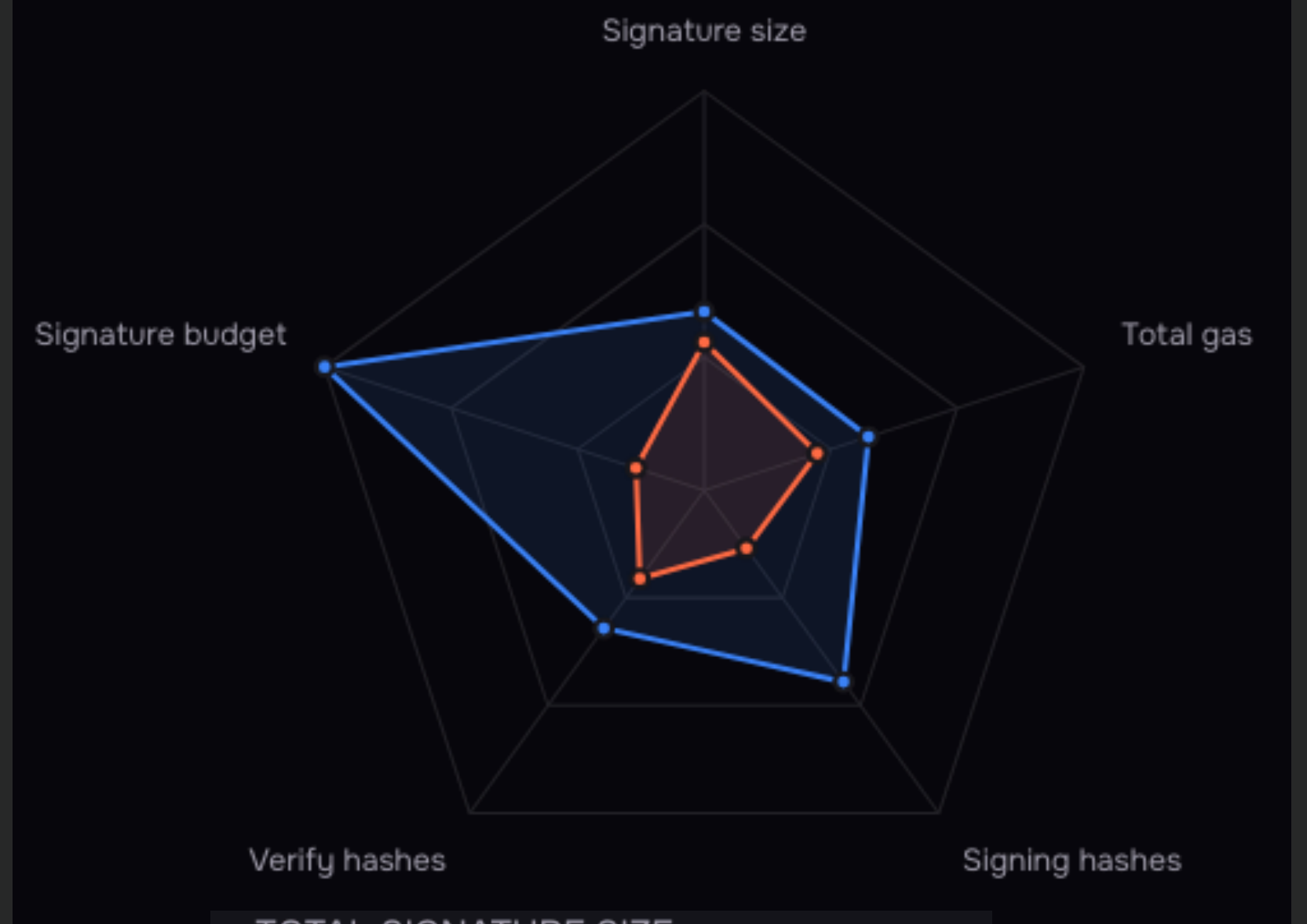
CALCULATED SCHEMATICS

$h = h' \times d = 4 \times 5 = 20 \cdot 2^{20}$ hypertree
leaf slots
 $len = 46 (43 + 3) \cdot \text{FORS digest } K \cdot A = 140 \text{ bits}$

Budget Limits ?

Min 2¹⁴

Max 2¹⁴



TOTAL SIGNATURE SIZE

6,576 B (6.4 KB)

- WOTS+ signatures 3.68 KB
- Hypertree auth 320 B
- FORS 2.56 KB
- Randomizer 16 B

Verification gas (est.)

212.98k

calldata
105,216 +
1,796 hashes
× 60

Signing cost (hashes)

37.26k

≈ 4 ms at 10
MH/s

Verification hashes

1,796

worst case ·

Public key

32 B

secret key 64
B

47.5s signature times
measured on a
ST33K1M5 secure
element (Ledger nano S+)



This is our SPHINCS variant (SPHINCS-G) signing on a Ledger Nano S Plus in ~1.5 seconds. 2^{24} signatures, 32-byte pubkey. All coming from our hash-based MPC work. We are overwhelmed by the support of legends within the Ethereum ecosystem and will share more soon!

*...signing on a Ledger
Nano S Plus in ~1.5
seconds. 2^{24} signatures,
32-byte pubkey.*





This is our SPHINCS variant (SPHINCS-G) signing on a Ledger Nano S Plus in ~1.5 seconds. 2^{24} signatures, 32-byte pubkey. All coming from our hash-based MPC work. We are overwhelmed by the support of legends within the Ethereum ecosystem and will share more soon!

*...signing on a Ledger
Nano S Plus in ~1.5
seconds. 2^{24} signatures,
32-byte pubkey.*





This is our SPHINCS variant (SPHINCS-G) signing on a Ledger Nano S Plus in ~1.5 seconds. 2^{24} signatures, 32-byte pubkey. All coming from our hash-based MPC work. We are overwhelmed by the support of legends within the Ethereum ecosystem and will share more soon!

...signing on a Ledger Nano S Plus in ~1.5 seconds. 2^{24} signatures, 32-byte pubkey.





The (Not-So-) Secret Sauce

★ The (Not-So-) Secret Sauce

- Realize that part of SPHINCS is public and can therefore be computed outside the secure element

★ The (Not-So-) Secret Sauce

- Realize that part of SPHINCS is **public** and can therefore be computed **outside the secure element**
- **Precompute** selected values **offline** ahead of the actual signing operation

★ The (Not-So-) Secret Sauce

- Realize that part of SPHINCS is **public** and can therefore be computed **outside the secure element**
- **Precompute** selected values **offline** ahead of the actual signing operation
- **Grind** at **signing time** by leveraging the externally computed public part

Path Forward

Path Forward

- **Formalize** and **analyze** the construction cryptographically (soon)

Path Forward

- **Formalize** and **analyze** the construction cryptographically (soon)
- **Experiment** with non-standard SPHINCS variants (e.g., WOTS+C / FORS+C)

Path Forward

- **Formalize** and **analyze** the construction cryptographically (soon)
- **Experiment** with non-standard SPHINCS variants (e.g., WOTS+C / FORS+C)
- **Native** account abstraction (Hegotà)

Path Forward

- **Formalize** and **analyze** the construction cryptographically (soon)
- **Experiment** with non-standard SPHINCS variants (e.g., WOTS+C / FORS+C)
- **Native** account abstraction (Hegotà)
- Formally verify the signature contracts in **Lean4**

Questions?

